UNIVERSIDADE ESTADUAL DO MARANHÃO
NÚCLEO DE TECNOLOGIAS PARA EDUCAÇÃO
CURSO SUPERIOR DE TECNOLOGIA EM ANÁLISE E DESENVOLVIMENTO DE SISTEMAS

ANDRÉ PEREIRA FERREIRA

ANÁLISE COMPARATIVA DE DESEMPENHO EM AMBIENTES DE PERSISTÊNCIA EM MEMÓRIA: o impacto da arquitetura Worker do FrankenPHP no framework CodeIgniter 4 frente ao ecossistema Laravel Octane.

Grajaú - MA
2026

ANDRÉ PEREIRA FERREIRA

ANÁLISE COMPARATIVA DE DESEMPENHO EM AMBIENTES DE PERSISTÊNCIA EM MEMÓRIA: o impacto da arquitetura Worker do FrankenPHP no framework CodeIgniter 4 frente ao ecossistema Laravel Octane.

Monografia de graduação apresentada ao curso de Tecnologia em Análise e Desenvolvimento de Sistemas da Universidade Estadual do Maranhão, Núcleo de Tecnologia para Educação, como requisito para a obtenção do grau de Tecnólogo.

Orientador: Prof. Me. Paulo Ericson Valentim Silva.

Grajaú - MA
2026

ANDRÉ PEREIRA FERREIRA

ANÁLISE COMPARATIVA DE DESEMPENHO EM AMBIENTES DE PERSISTÊNCIA EM MEMÓRIA: o impacto da arquitetura Worker do FrankenPHP no framework CodeIgniter 4 frente ao ecossistema Laravel Octane.

Monografia de graduação apresentada ao curso de Tecnologia em Análise e Desenvolvimento de Sistemas da Universidade Estadual do Maranhão, Núcleo de Tecnologia para Educação, como requisito para a obtenção do grau de Tecnólogo.

Aprovado em: ____ / ____ / _____.

BANCA EXAMINADORA

_____
Prof. Me. Paulo Ericson Valentim Silva (Orientador)
Universidade Estadual do Maranhão

_____
[Nome e titulação do(a) examinador(a) 1]
[Instituição]

_____
[Nome e titulação do(a) examinador(a) 2]
[Instituição]

Ao Senhor meu Deus; à minha esposa, Jéssica;
e aos meus familiares, que depositam em mim
uma confiança maior do que julgo merecer.

AGRADECIMENTOS

Aos meus amigos Emerson, Rogério e Raone, pelos estudos compartilhados, pelas dicas, pelos projetos e pelo incentivo ao longo dos últimos anos. À Universidade Estadual do Maranhão (UEMA), pela formação e pelo direito ao ensino público e de qualidade.

RESUMO

Este trabalho analisa o impacto da execução persistente em memória no desempenho do framework CodeIgniter 4 (CI4). Compararam-se o PHP-FPM, o FrankenPHP em modo clássico e o FrankenPHP em modo Worker, com o Laravel Octane como referência de mercado, em contêineres Docker de mesma base (Debian 13) e com 1,0 CPU e 512 MB cada, sob carga gerada pelo k6 (100 usuários virtuais, 15 minutos) e monitoramento por Prometheus e Grafana. Em três execuções por cenário, a vazão média foi de 143 requisições por segundo (PHP-FPM), 156 (FrankenPHP clássico), 304 (CI4 em modo Worker) e 337 (Laravel Octane), medidas no patamar de 10 minutos e sem erros. O modo Worker mais que dobrou a vazão do CI4 (+112%) e reduziu à metade as latências p95 e p99, ao custo de 46 MiB adicionais de memória, enquanto o modo clássico trouxe ganho modesto (+9%). Verificou-se que o CI4 em modo Worker preserva o estado da aplicação e a conexão com o banco, como o Octane, e que a memória permaneceu estável em testes de 30 minutos. Conclui-se que o modo Worker compensa em vazão e latência para o CI4, aproximando-o do Laravel Octane.

Palavras-chave: CodeIgniter 4; FrankenPHP; desempenho; benchmarking; PHP.

ABSTRACT

This work analyzes the impact of persistent in-memory execution on the performance of the CodeIgniter 4 (CI4) framework. PHP-FPM, FrankenPHP in classic mode and FrankenPHP in Worker mode were compared, with Laravel Octane as a market reference, in Docker containers sharing the same base image (Debian 13) and limited to 1.0 CPU and 512 MB each, under load generated by k6 (100 virtual users, 15 minutes) and monitored with Prometheus and Grafana. Over three runs per scenario, mean throughput at the 10-minute plateau was 143 requests per second (PHP-FPM), 156 (classic FrankenPHP), 304 (CI4 in Worker mode) and 337 (Laravel Octane), with no errors. Worker mode more than doubled CI4 throughput (+112%) and halved p95 and p99 latency, at the cost of 46 MiB of additional memory, whereas classic mode brought a modest gain (+9%). CI4 in Worker mode was found to preserve application state and the database connection, like Octane, and memory remained stable in 30-minute tests. It is concluded that Worker mode pays off in throughput and latency for CI4, bringing it close to Laravel Octane.

Keywords: CodeIgniter 4; FrankenPHP; performance; benchmarking; PHP.

LISTA DE ILUSTRAÇÕES

Quadro 1 – Estado preservado entre requisições, por cenário.....20
Figura 1 – Vazão média (RPS) por cenário, com desvio-padrão das 3 execuções.....22
Figura 2 – Latência p95 por endpoint e cenário.....23
Figura 3 – CPU e RAM médias por cenário.....25
Figura 4 – Memória dos contêineres C e D ao longo do patamar de 30 minutos.....26
Figura 5 – Painel do Grafana, cenário A (execução de captura).....27
Figura 6 – Painel do Grafana, cenário B (execução de captura).....28
Figura 7 – Painel do Grafana, cenário C (execução de captura).....29
Figura 8 – Painel do Grafana, cenário D (execução de captura).....30
Figura 9 – Painel do k6, cenário A (execução de captura).....31
Figura 10 – Painel do k6, cenário B (execução de captura).....32
Figura 11 – Painel do k6, cenário C (execução de captura).....33
Figura 12 – Painel do k6, cenário D (execução de captura).....34

LISTA DE TABELAS

Tabela 1 – Vazão e latência global por cenário (média das 3 execuções).....21
Tabela 2 – Comparações par a par (testes t de Welch, n = 3).....23
Tabela 3 – Latência por endpoint (média / p95, em ms).....23
Tabela 4 – Consumo de recursos no patamar sustentado (média das 3 execuções).....24
Tabela 5 – Testes longos (patamar de 30 minutos) nos cenários C e D.....26
Tabela 6 – Resultados de cada uma das 15 execuções de 15 minutos.....69
Tabela 7 – Resultados dos testes longos de 30 minutos de patamar.....69

LISTA DE ABREVIATURAS E SIGLAS

ANOVA	Analysis of Variance (análise de variância)
API	Application Programming Interface
CI4	CodeIgniter 4
CPU	Central Processing Unit (unidade central de processamento)
HTTP	HyperText Transfer Protocol
NTS	Non-Thread-Safe
PHP-FPM	PHP FastCGI Process Manager
RAM	Random Access Memory (memória de acesso aleatório)
REST	Representational State Transfer
RPS	Requisições por segundo (requests per second)
VU	Virtual User (usuário virtual)
ZTS	Zend Thread Safety

SUMÁRIO

1 INTRODUÇÃO.....12
1.1 Problema de pesquisa.....13
1.2 Objetivo geral.....13
1.3 Objetivos específicos.....13
2 FUNDAMENTAÇÃO TEÓRICA.....14
2.1 Modelos de execução do PHP.....14
2.2 FrankenPHP: modo clássico e modo Worker.....14
2.3 Laravel Octane e suporte do CodeIgniter 4 ao modo Worker.....15
2.4 Avaliação de desempenho de frameworks web.....15
2.5 Rigor em benchmarks.....16
3 METODOLOGIA.....17
3.1 Objeto de estudo e cenários.....17
3.2 Endpoints de teste.....18
3.3 Instrumentação.....18
3.4 Procedimento de coleta.....19
3.5 Análise dos dados.....19
4 RESULTADOS E DISCUSSÃO.....20
4.1 Validação do ambiente e persistência de estado.....20
4.2 Vazão (requisições por segundo) e latência.....21
4.3 Latência por endpoint.....23
4.4 Consumo de CPU e memória.....24
4.5 Estabilidade da memória nos testes longos.....25
4.6 Visão temporal nas execuções de captura.....26
4.7 Discussão.....34
4.8 Ameaças à validade e limitações.....37
5 CONSIDERAÇÕES FINAIS.....38
REFERÊNCIAS.....41
APÊNDICE A – ORQUESTRAÇÃO DOS CONTÊINERES (DOCKER-COMPOSE.YML).....42
APÊNDICE B – IMAGENS E CONFIGURAÇÃO DOS SERVIDORES.....47
APÊNDICE C – SCRIPTS DE CARGA DO K6.....52
APÊNDICE D – SCRIPTS DE AUTOMAÇÃO E ANÁLISE.....56
APÊNDICE E – CONTROLADORES DOS ENDPOINTS.....66
APÊNDICE F – RESULTADOS POR EXECUÇÃO.....69
APÊNDICE G – AMBIENTE DE EXECUÇÃO.....70

1 INTRODUÇÃO

A evolução constante das tecnologias web tem exigido que sistemas modernos ofereçam alta escalabilidade e baixo tempo de resposta, forçando desenvolvedores a tomarem decisões estratégicas sobre a arquitetura de execução. Historicamente, a linguagem PHP dominou o cenário do desenvolvimento *server-side* operando no modelo *Request-Response-Die*, no qual o interpretador é iniciado, o *script* executado e todos os recursos destruídos ao final de cada requisição HTTP. Embora o PHP-FPM tenha trazido avanços ao manter processos persistentes, ele ainda opera de forma sequencial e bloqueante, o que gera um *overhead* de *bootstrapping* (inicialização do *framework*) que se repete a cada acesso, limitando o desempenho em cenários de alta carga.

A justificativa para a realização deste *benchmark* reside na necessidade de validar a transição desse modelo tradicional para o paradigma de memória persistente por meio de servidores de aplicação modernos. O surgimento de tecnologias como o Swoole e o FrankenPHP permitiu que o PHP funcionasse com laços de eventos persistentes, mantendo a aplicação carregada na memória RAM e reduzindo a inicialização. O ecossistema Laravel, por meio do Laravel Octane, já consolidou essa prática como padrão de mercado para alto desempenho, tratando nativamente a limpeza de estado e a injeção de dependências. Ainda assim, estudos que comparam a arquitetura tradicional (PHP-FPM com servidor web separado) com a integrada (FrankenPHP) são escassos: a literatura concentra-se na otimização no nível da aplicação ou do *framework*, e os estudos sobre a camada do servidor de aplicação são limitados (DHANISWARA; KUSUMASARI; ALAM, 2025).

Entretanto, o *framework* CodeIgniter 4 (CI4), embora amplamente utilizado pela sua leveza e simplicidade, passou a oferecer suporte nativo ao modo Worker do FrankenPHP apenas a partir da versão 4.7. Até então era possível utilizar o modo clássico, mas sem o mesmo desempenho. Para a comunidade de desenvolvimento web, é relevante compreender se adotar o modo Worker no CI4 justifica os ganhos de vazão (RPS) e latência, ou se o uso do FrankenPHP em seu modo clássico já seria suficiente para superar o legado do PHP-FPM.

A relevância deste estudo é multifacetada:

a) eficiência de recursos: em infraestrutura em nuvem, onde o consumo de CPU e RAM impacta diretamente os custos operacionais, validar arquiteturas que otimizam o uso do *hardware* é uma prioridade estratégica;
b) modernização de sistemas legados: muitos sistemas baseados em CodeIgniter precisam escalar sem reescrita completa para outros *frameworks* ou linguagens;

c) adoção de novas ferramentas: o FrankenPHP é uma tecnologia recente que integra o PHP a um servidor escrito em Go, e este estudo fornece dados empíricos sobre seu desempenho em diferentes modos de operação.

1.1 Problema de pesquisa

Em que medida a execução persistente em memória do FrankenPHP (modo Worker) altera a vazão, a latência e o consumo de recursos do CodeIgniter 4, em comparação com o PHP-FPM e com o FrankenPHP em modo clássico, tendo o Laravel Octane como referência?

1.2 Objetivo geral

Comparar o desempenho do CodeIgniter 4 sob PHP-FPM, FrankenPHP em modo clássico e FrankenPHP em modo Worker, utilizando o Laravel Octane como referência, sob carga controlada e com o mesmo orçamento de recursos.

1.3 Objetivos específicos

a) montar um ambiente experimental conteinerizado, reprodutível, com limites idênticos de CPU e memória para os quatro cenários;
b) medir e comparar vazão (requisições por segundo) e latência (p50 e p95) dos cenários sob carga crescente e sustentada;
c) avaliar o consumo de CPU e memória de cada cenário durante o teste;
d) verificar quais elementos (estado da aplicação e conexão com o banco) são efetivamente preservados entre requisições em cada arquitetura;
e) discutir se o ganho observado justifica a complexidade de adoção do modo Worker em aplicações CodeIgniter 4.

2 FUNDAMENTAÇÃO TEÓRICA

2.1 Modelos de execução do PHP

No modelo tradicional, o PHP processa cada requisição de forma independente: o interpretador carrega o código, executa-o e libera os recursos ao final, sem preservar estado em memória entre requisições. O PHP-FPM (*FastCGI Process Manager*) mantém um *pool* de processos já iniciados, o que evita criar um processo a cada acesso, mas o código da aplicação, e portanto a inicialização do *framework*, continua sendo executado do zero a cada requisição. Luz, Silva e Emer (2024) descrevem esse modelo como síncrono e sem estado (*stateless*) e o comparam ao modelo assíncrono da extensão Swoole, em experimento de carga com análise estatística (ANOVA e teste t), concluindo que o modo de operação foi o fator determinante do desempenho, com o modelo assíncrono superando o síncrono no atendimento a múltiplas requisições.

Dhaniswara, Kusumasari e Alam (2025) detalham a origem desse custo na arquitetura usual de implantação, em que um servidor web (como o Nginx) atua como *proxy* reverso e encaminha as requisições ao PHP-FPM pelo protocolo FastCGI. A separação entre os dois processos dá flexibilidade e estabilidade, mas acrescenta uma comunicação entre processos a cada requisição, além da reinicialização da aplicação, o que, segundo os autores, limita o desempenho máximo, sobretudo em aplicações de grande porte.

2.2 FrankenPHP: modo clássico e modo Worker

O FrankenPHP é um servidor de aplicação para PHP construído sobre o servidor Caddy, que incorpora o interpretador PHP ao próprio processo do servidor (DUNGLAS, 2024). Sem nenhuma configuração adicional, opera no **modo clássico**, em que funciona como um servidor PHP tradicional, servindo diretamente os arquivos PHP, e pode substituir o PHP-FPM ou o Apache com `mod_php`. O número de conexões aceitas e enfileiradas é limitado apenas pelos recursos do sistema, e o PHP usa um número fixo de *threads* iniciadas na partida, comparável ao modo estático do PHP-FPM (DUNGLAS, 2024). Por padrão, o servidor inicia o dobro de *threads* em relação ao número de núcleos de CPU disponíveis, e a diretiva `num_threads` permite fixar outro valor (DUNGLAS, 2024).

No **modo Worker**, o *script* de entrada da aplicação é carregado uma única vez e permanece vivo, atendendo requisições em um laço: a inicialização do *framework* ocorre uma vez por *worker*, e cada requisição executa apenas o tratamento do pedido (DUNGLAS, 2024).

A contrapartida é que o programador passa a ser responsável por não vazar estado de uma requisição para outra, já que objetos e variáveis estáticas podem sobreviver entre elas.

Dhaniswara, Kusumasari e Alam (2025) descrevem essa abordagem como um modelo integrado, em que as funções do servidor web e do interpretador PHP são reunidas em um único processo, o que elimina a comunicação entre processos do modelo tradicional, com o risco de falha centralizada. No modo Worker, o *framework* é inicializado uma vez e mantém seu estado em memória, em vez de recarregar todos os componentes a cada requisição. Os autores ressaltam que a comparação direta entre os modelos tradicional e integrado ainda é pouco estudada, pois o FrankenPHP e tecnologias semelhantes surgiram por volta de 2022 e 2023.

2.3 Laravel Octane e suporte do CodeIgniter 4 ao modo Worker

O Laravel Octane é o pacote oficial do Laravel para executar a aplicação sobre servidores persistentes, entre eles FrankenPHP, Swoole e RoadRunner (OTWELL, 2024). Ele mantém o *framework* inicializado em memória entre requisições e trata a limpeza do estado por requisição, o que o torna a referência de mercado para esse modelo no ecossistema PHP. O CodeIgniter 4 passou a oferecer suporte nativo ao modo Worker a partir da versão 4.7, com um *script* de entrada gerado pelo próprio *framework* (CODEIGNITER FOUNDATION, 2024), que mantém o *framework* carregado e trata cada requisição em um laço. O que esse *script* preserva ou reinicia entre requisições é verificado experimentalmente na Seção 4.1. Quanto à implantação, Dhaniswara, Kusumasari e Alam (2025) executaram o Laravel Octane sobre o FrankenPHP em contêiner Docker, a partir da imagem oficial do FrankenPHP, o que dispensa um contêiner separado para o Nginx e reduz o número de componentes da implantação.

2.4 Avaliação de desempenho de frameworks web

Estudos recentes comparam *frameworks* web quanto a tempo de resposta, vazão e uso de recursos com ferramentas de carga como o k6. Azzahidi, Wijayanto e Darmawan (2025), por exemplo, avaliaram Spring Boot, Flask, Express.js, Laravel com FrankenPHP e Gin em APIs REST e destacaram o Laravel com FrankenPHP pelo uso eficiente de memória. O k6, mantido pela Grafana Labs, permite descrever cenários de carga em JavaScript, definir rampas de usuários virtuais e exportar métricas para sistemas de monitoramento como o Prometheus (GRAFANA LABS, 2024), e é a ferramenta adotada neste trabalho.

O estudo mais próximo deste trabalho é o de Dhaniswara, Kusumasari e Alam (2025), que compararam duas arquiteturas de implantação de uma aplicação Laravel 12 em contêineres

Docker com limites idênticos de recursos: PHP-FPM com Nginx e FrankenPHP em modo Worker com Laravel Octane. Com o Apache JMeter (100 usuários virtuais por 10 minutos), mediram tempo de resposta, vazão e uso de CPU e memória. Em um *endpoint* de carga leve por requisição, o FrankenPHP teve tempo de resposta médio cerca de 15,8% menor e vazão 18,6% maior, com percentis mais estáveis, ao custo de maior consumo de memória; em um *endpoint* que serializa grande volume de dados, as duas arquiteturas atingiram o limite de CPU e tiveram desempenho semelhante, o que os autores atribuem à aplicação, e não ao servidor. Eles também apontam como lacuna a escassez de estudos quantitativos sobre a camada do servidor de aplicação, já que a literatura se concentra na otimização da aplicação ou do *framework*. Este trabalho contribui para essa lacuna no caso do CodeIgniter 4, com o modo clássico do FrankenPHP como cenário adicional e o k6 como ferramenta de carga.

2.5 Rigor em benchmarks

A literatura sobre *benchmarking* ressalta que resultados de desempenho dependem do desenho experimental e do ambiente de execução. Laaber, Basmaci e Salza (2021) tratam da instabilidade de *benchmarks* de *software*, isto é, da variabilidade de medições repetidas, o que justifica repetir as execuções e relatar a dispersão. Zhan (2022) discute a necessidade de *benchmarks* confiáveis, representativos e reprodutíveis para tecnologias computacionais emergentes; Brunst et al. (2022) relatam a experiência de construir e aplicar um conjunto padronizado de *benchmarks* (SPEChpc 2021); e Jia et al. (2013) mostram que a diversidade de aplicações e de volumes de dados afeta as conclusões de um *benchmark*. Essas contribuições orientam as escolhas metodológicas deste estudo: ambiente conteinerizado com limites idênticos de recursos, três execuções por cenário em ordem rotativa, relato de desvio-padrão e discussão explícita das limitações da carga sintética utilizada.

3 METODOLOGIA

A presente pesquisa caracteriza-se como um estudo experimental e quantitativo, de natureza comparativa, que mede o desempenho de diferentes arquiteturas de execução para aplicações PHP em condições controladas. Esta seção atualiza e detalha a Metodologia apresentada na etapa anterior, com as versões efetivamente utilizadas e o procedimento de coleta executado.

3.1 Objeto de estudo e cenários

Foram avaliados quatro cenários principais, todos com o mesmo orçamento de recursos, de 1,0 CPU e 512 MB de RAM por cenário (no cenário A, dividido entre PHP-FPM, com 0,85 CPU e 448 MB, e Nginx, com 0,15 CPU e 64 MB), e um cenário complementar de sensibilidade:

a) **Cenário A (legado):** CodeIgniter 4.7.5 sobre PHP 8.3.35-FPM, com 4 processos estáticos, e Nginx 1.31.7 como servidor web;

b) **Cenário B (clássico moderno):** o mesmo aplicativo sobre FrankenPHP 1.13.0 em modo clássico, como substituto direto do PHP-FPM, com 4 *threads* PHP, número igual ao de processos do cenário A e de *workers* dos cenários C e D;

c) **Cenário C (vanguarda):** o mesmo aplicativo sobre FrankenPHP em modo Worker, com 4 *workers* e o suporte nativo do CodeIgniter 4.7 ao modo Worker (arquivo `public/frankenphp-worker.php`, gerado pelo comando `php spark worker:install` do próprio *framework*);

d) **Cenário D (referência):** Laravel 13.36.0 com Laravel Octane 2.22.0 sobre FrankenPHP, com 4 *workers*;

e) **Cenário E (sensibilidade):** idêntico ao B, porém sem a diretiva `num_threads`, de modo que o FrankenPHP usa o número padrão de *threads*, o dobro dos núcleos de CPU que ele enxerga. Com o limite de 1,0 CPU do contêiner, isso resultou em 2 *threads* PHP (contra 4 no B), valor confirmado no registro de inicialização do servidor (`total_threads`: 2); sem o limite, o servidor iniciaria 32 *threads* (2 × 16 *threads* lógicas do hospedeiro). Foi incluído para verificar se o resultado do modo clássico depende dessa configuração.

Todas as imagens de aplicação e de servidor web dos cenários A a E têm a mesma base, Debian 13 (*trixie*), com a mesma versão da biblioteca C (glibc 2.41), e o mesmo PHP 8.3.35, o

que evita que diferenças de sistema operacional contaminem a comparação. A única diferença de compilação é inerente às arquiteturas: o PHP do FrankenPHP é compilado em modo *thread-safe* (ZTS), exigido por ele, e o do PHP-FPM em modo *non-thread-safe* (NTS). As imagens e os pacotes foram fixados por versão e por *digest*. Todos usam OPcache com a mesma configuração e o mesmo banco PostgreSQL 16 (contêiner separado, com 0,5 CPU e 256 MB, fora do orçamento dos cenários e idêntico em todos eles). Esse desenho, com limites idênticos de recursos por arquitetura, a mesma configuração de OPcache e o uso de contêineres Docker para obter condições consistentes e isoladas, segue o adotado por Dhaniswara, Kusumasari e Alam (2025) na comparação entre PHP-FPM com Nginx e FrankenPHP.

3.2 Endpoints de teste

Os cenários expõem os mesmos endpoints: `/health` (resposta mínima, mede a latência base), `/compute` (gera e serializa em JSON 1.000 elementos, com carga de CPU), `/memory` (incrementa um contador estático e retorna o PID, demonstrando persistência de estado) e `/db` (consulta por chave primária a uma tabela com 10.000 linhas, expondo o PID da conexão PostgreSQL).

3.3 Instrumentação

A carga foi gerada pelo Grafana k6 2.3.0 em contêiner isolado (2,0 CPU e 512 MB). Cada execução tem 15 minutos: 2 min de aquecimento até 10 usuários virtuais, 2 min de rampa até 100, 10 min de patamar a 100 usuários e 1 min de desaceleração. Cada usuário dispara requisições continuamente, sem espera entre elas, com a distribuição 10% `/health`, 30% `/compute`, 30% `/memory` e 30% `/db`. As métricas foram exportadas ao Prometheus e visualizadas no Grafana, e o consumo de CPU e RAM dos contêineres foi amostrado com `docker stats` a cada poucos segundos, em paralelo ao teste. A vazão e a latência são calculadas apenas sobre o patamar de 10 minutos (aquecimento, rampa e descida excluídos), e a vazão é o número de requisições do patamar dividido por 600 s; a CPU e a RAM são médias da mesma janela. Como a carga usa um único nível de concorrência (100 usuários virtuais), a vazão medida é a observada nessa carga, e não necessariamente a capacidade máxima de cada cenário. As métricas escolhidas (vazão, tempo de resposta e uso de CPU e memória) são as mesmas usadas por Dhaniswara, Kusumasari e Alam (2025) na avaliação de arquiteturas de implantação em contêineres, que também reportam percentis (p50, p90, p95 e p99) do tempo de resposta.

3.4 Procedimento de coleta

A coleta foi automatizada por *script* e executada de uma só vez, do início ao fim, em 19 de outubro de 2026, sem outra ferramenta concorrendo pelos recursos do hospedeiro nas execuções de medição (o navegador só foi usado na etapa (v), para as capturas). As etapas foram: (i) diagnóstico de persistência por *worker* nos cenários C e D (200 requisições sequenciais e 200 concorrentes a `/memory` e `/db`, com 1 e com 4 *workers*); (ii) três execuções de cada cenário A, B, C e D, em ordem rotativa (A, B, C, D, A, B, C, D, A, B, C, D), para reduzir o viés de ordem e de temperatura do hospedeiro; (iii) três execuções do cenário E; (iv) um teste longo, com patamar de 30 minutos, nos cenários C e D, para observar a estabilidade da memória; e (v) uma execução adicional de cada cenário, de 15 minutos, dedicada à captura dos painéis do Grafana. Antes de cada execução, os contêineres anteriores eram derrubados, o *cache* do sistema operacional era limpo (`drop_caches`) e o volume do Prometheus era removido, de modo que cada execução começa sem dados de execuções anteriores.

Os experimentos rodaram em um notebook com processador AMD Ryzen 7 5700U (8 núcleos e 16 *threads*) e 14 GiB de RAM, com Linux e Docker 28.1.1 (Apêndice G). Os *scripts* de coleta e de análise estão no Apêndice D.

3.5 Análise dos dados

Para cada cenário foram calculadas a média e o desvio-padrão, entre as três execuções, da vazão (RPS), da latência (p50, p95 e p99), da taxa de erros e do consumo de CPU e RAM, todos calculados no patamar sustentado de 10 minutos. Os ganhos foram expressos em variação percentual. Como as execuções de um mesmo cenário foram apenas três, aplicou-se, em complemento, a análise de variância de um fator (ANOVA) e testes t de Welch par a par, com correção de Bonferroni, em linha com o procedimento adotado por Luz, Silva e Emer (2024); com n = 3 o poder desses testes é limitado, e eles servem como reforço, e não como único critério. A interpretação da latência considera a Lei de Little para sistemas fechados (LITTLE, 1961). O processamento é feito pelo *script* `scripts/analyze.py` (Apêndice D), que gera as tabelas e figuras do capítulo de Resultados.

4 RESULTADOS E DISCUSSÃO

Este capítulo apresenta os resultados obtidos nos cenários descritos na Metodologia e os interpreta à luz do referencial teórico. Os valores oficiais de vazão e latência são do patamar de 10 minutos de cada execução (aquecimento, rampa e descida excluídos) e provêm de 15 execuções do teste de carga de 15 minutos (3 repetições de cada um dos cenários A, B, C, D e E) e de dois testes longos de 30 minutos de patamar (cenários C e D). As figuras extraídas do Grafana e do painel do k6 (Figuras 5 a 12) vêm de uma execução adicional de cada cenário, feita apenas para captura, e não entram nas médias.

4.1 Validação do ambiente e persistência de estado

Em todas as execuções válidas, as verificações (*checks*) do k6 passaram e a taxa de falhas HTTP foi de 0,00%, o que indica que as diferenças de desempenho adiante não decorrem de erros de aplicação.

Os endpoints `/memory` e `/db` revelam o que cada arquitetura realmente preserva entre requisições. O Quadro 1 resume as verificações feitas: cinco requisições sequenciais em cada cenário A, B e E, e 200 requisições sequenciais e 200 concorrentes nos cenários C e D, com 1 e com 4 *workers*.

Quadro 1 – Estado preservado entre requisições, por cenário

Cenário	**`/memory`: contador**	**`/db`: conexão PostgreSQL**
A – CI4 + PHP-FPM	sempre 1; PID do processo varia (8, 10, 8, 10, 8)	nova a cada requisição (PIDs 40 a 44)
B e E – CI4 + FrankenPHP clássico	sempre 1; PID 1	nova a cada requisição (PIDs 40 a 44)
C – CI4 + FrankenPHP Worker	cresce (2 a 201 em um *worker*); cada *worker* tem o seu contador	mantida: um único PID por *worker* (um *worker*: PID 70; quatro: PIDs 40, 48, 49 e 50)
D – Laravel Octane	cresce (2 a 201 em um *worker*); cada *worker* tem o seu contador	mantida: um único PID por *worker*

Fonte: elaborado pelo próprio autor (2026), a partir de `results/diagnostico`.

O Quadro 1 mostra que os dois cenários em modo Worker (C e D) preservam tanto o estado da aplicação quanto a conexão com o banco, ao passo que A, B e E reiniciam ambos a cada requisição. Com quatro *workers*, os pedidos concorrentes são distribuídos entre quatro processos persistentes, cada um com seu contador e sua conexão, de modo que o contador global não é compartilhado entre eles. No cenário D com quatro *workers*, o diagnóstico registrou cinco identificadores de *worker* e, com um *worker*, dois; o padrão (um *worker*

encerrado após cerca de 500 requisições e substituído por outro) é coerente com o reciclo periódico de processos que o Laravel Octane faz por padrão, e não ocorreu no CodeIgniter 4. Dhaniswara, Kusumasari e Alam (2025) também configuraram o Octane para reiniciar cada *worker* após 500 requisições (`--max-requests=500`), equiparando-o ao reciclo de processos do PHP-FPM (`pm.max_requests`). A diferença entre os dois comportamentos é relevante para a memória, tratada na Seção 4.5.

4.2 Vazão (requisições por segundo) e latência

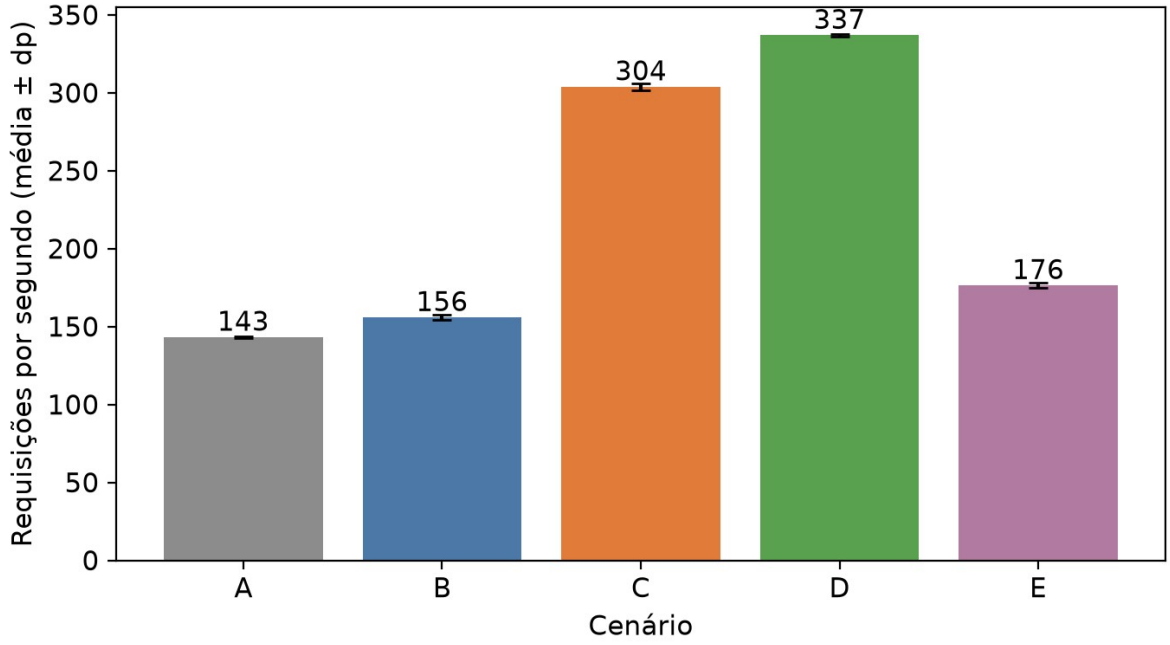
A Tabela 1 e a Figura 1 apresentam a vazão média e o desvio-padrão das três execuções de cada cenário, medidas no patamar de 10 minutos sob 100 usuários virtuais simultâneos (a vazão observada nessa carga, não necessariamente a capacidade máxima de cada cenário), em um orçamento de 1,0 CPU e 512 MB por cenário, e a latência global das requisições.

Tabela 1 – Vazão e latência global por cenário (média das 3 execuções)

Cenário	**RPS (média ± dp)**	**Latência p50 (ms)**	**Latência p95 (ms)**	**Latência p99 (ms)**	**Erros (%)**
A – CI4 + PHP-FPM	143,2 ± 0,6	698,2	789,7	807,2	0,00
B – CI4 + FrankenPHP clássico	155,8 ± 1,6	618,0	743,2	793,2	0,00
E – CI4 + FrankenPHP clássico (2 *threads*, padrão)	176,4 ± 1,8	563,3	676,1	734,3	0,00
C – CI4 + FrankenPHP Worker	303,9 ± 2,1	309,8	397,9	408,3	0,00
D – Laravel Octane	336,9 ± 1,0	298,2	383,6	399,7	0,00

Fonte: elaborado pelo próprio autor (2026), a partir dos resumos do k6. Nota: RPS (requisições do patamar divididas por 600 s) e percentis calculados no patamar de 10 minutos, sem aquecimento, rampa e descida.

Figura 1 – Vazão média (RPS) por cenário, com desvio-padrão das 3 execuções



Fonte: elaborado pelo próprio autor (2026).

Em relação ao cenário A, o modo clássico do FrankenPHP (B) apresentou vazão 8,8% maior, o Worker com CodeIgniter 4 (C) apresentou vazão 112,2% maior (cerca de 2,1 vezes) e o Laravel Octane (D) apresentou vazão 135,2% maior (cerca de 2,4 vezes). Entre C e B, que usam o mesmo servidor e o mesmo número de unidades de concorrência, a diferença é de 95,0% a favor de C. A latência acompanha a vazão: o p95 do cenário C é 49,6% menor que o do A (397,9 contra 789,7 ms) e o p99, 49,4% menor (408,3 contra 807,2 ms). O desvio-padrão entre execuções do mesmo cenário ficou entre 0,3% e 1,0% da média, muito menor que as diferenças entre cenários.

A Tabela 2 reúne os testes estatísticos. A análise de variância (ANOVA) indicou diferença entre os cinco cenários na vazão ($F = 10.524,5$; $p < 0,001$), no p95 ($F = 550,7$; $p < 0,001$) e no p99 ($F = 855,2$; $p < 0,001$). Nos testes t de Welch par a par, com correção de Bonferroni para as seis comparações, todas as diferenças de vazão permaneceram significativas ao nível de 5%, inclusive a de D sobre C (+10,9%; p ajustado = 0,001). No p95, as diferenças de B sobre A (p ajustado = 0,65) e de E sobre B (0,24) não foram significativas, e no p99 tampouco (B sobre A, p ajustado = 0,76; E sobre B, 0,23). Com apenas três execuções por cenário, o poder dos testes é limitado e a normalidade não pode ser verificada; por isso os valores de p devem ser lidos como reforço à diferença de ordem de grandeza entre os cenários, e não como critério isolado.

Tabela 2 – Comparações par a par (testes t de Welch, n = 3)

Comparação	Vazão (%)	p ajust.	p95 (%)	p ajust.	p99 (%)	p ajust.
B contra A	+8,8	0,013	−5,9	0,65	−1,7	0,76
E contra B	+13,2	0,001	−9,0	0,24	−7,4	0,23
C contra A	+112,2	< 0,001	−49,6	< 0,001	−49,4	< 0,001
C contra B	+95,0	< 0,001	−46,5	0,014	−48,5	0,001
C contra E	+72,3	< 0,001	−41,1	0,004	−44,4	0,011
D contra C	+10,9	0,001	−3,6	0,003	−2,1	0,037

Fonte: elaborado pelo próprio autor (2026), com `scripts/analyze.py`. Nota: variações percentuais da média do primeiro cenário em relação ao segundo; p ajustado pelo método de Bonferroni.

4.3 Latência por endpoint

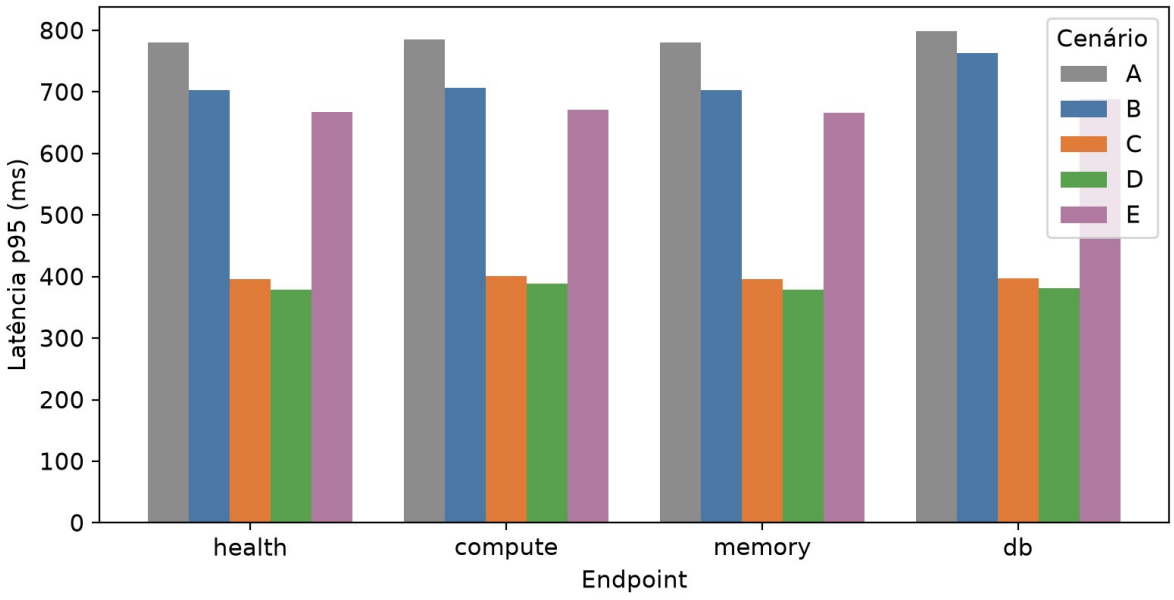
A Tabela 3 e a Figura 2 detalham a latência média e o percentil 95 por endpoint, também no patamar.

Tabela 3 – Latência por endpoint (média / p95, em ms)

Cenário	/health	/compute	/memory	/db
A – CI4 + PHP-FPM	684,2 / 780,2	691,3 / 784,7	683,7 / 780,5	721,3 / 798,2
B – CI4 + FrankenPHP clássico	625,3 / 702,9	634,2 / 706,4	625,0 / 702,6	669,2 / 763,2
E – CI4 + FrankenPHP clássico (2 *threads*, padrão)	558,7 / 667,2	562,9 / 670,6	558,9 / 666,5	579,4 / 688,5
C – CI4 + FrankenPHP Worker	324,3 / 396,1	336,2 / 400,4	324,1 / 396,2	326,1 / 396,7
D – Laravel Octane	291,6 / 379,1	304,4 / 388,9	291,4 / 378,6	294,1 / 381,2

Fonte: elaborado pelo próprio autor (2026).

Figura 2 – Latência p95 por endpoint e cenário



Fonte: elaborado pelo próprio autor (2026).

Um achado importante é que a latência é praticamente a mesma nos quatro endpoints dentro de cada cenário, embora o `/health` não faça processamento e o `/compute` gere e serialize mil registros. A explicação é a forma de carga: o k6 mantém 100 usuários virtuais disparando requisições continuamente, sem tempo de espera entre elas, e o servidor, limitado a 1 CPU, está saturado (Seção 4.4). Nessa condição, o tempo de resposta é dominado pelo tempo de fila e, pela Lei de Little (LITTLE, 1961), a latência média em um sistema fechado aproxima-se de usuários ÷ vazão. A conta confere: 100 ÷ 143,2 ≈ 698 ms no cenário A (latência média medida: 697 ms), 100 ÷ 303,9 ≈ 329 ms no cenário C (328 ms) e 100 ÷ 336,9 ≈ 297 ms no cenário D (296 ms). Consequentemente, a latência aqui é um espelho da vazão, e não uma medida independente do custo de cada endpoint. A diferença entre `/db` e `/health` é pequena em todos os cenários: 37 ms no cenário A, 44 ms no B, 21 ms no E, 2 ms no C e 2,5 ms no D. Ela é maior nos cenários que abrem uma nova conexão a cada requisição (A, B, E) do que nos que a reaproveitam (C e D), o que é coerente com o Quadro 1, mas o desenho do experimento não isola o custo de cada endpoint com precisão. Para isso seria necessária carga com taxa de chegada fixa (modelo aberto), tema indicado nos trabalhos futuros.

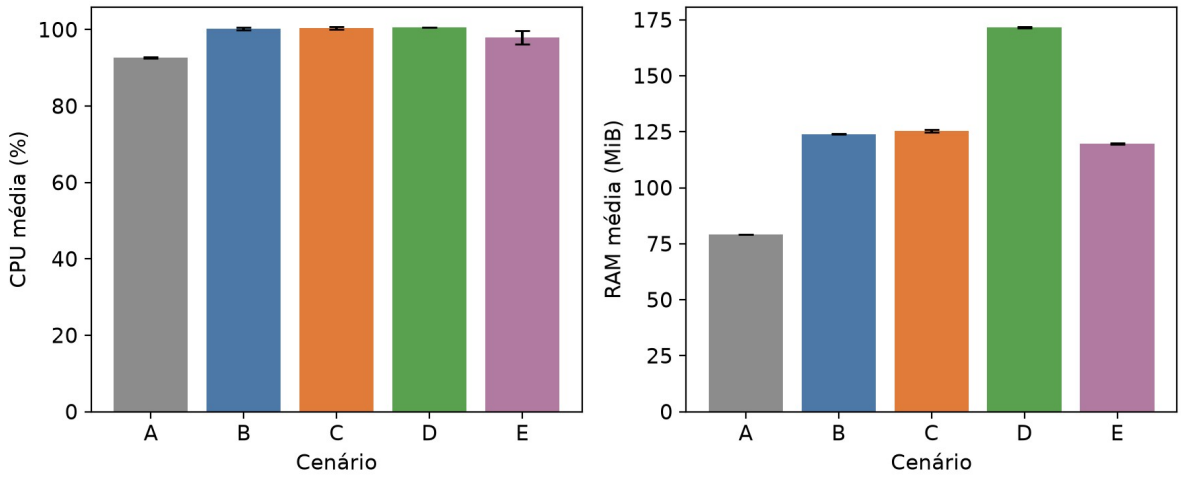
4.4 Consumo de CPU e memória

Tabela 4 – Consumo de recursos no patamar sustentado (média das 3 execuções)

Cenário	**CPU média (% de 1 núcleo)**	**RAM média (MiB)**	**RAM pico (MiB)**
A – CI4 + PHP-FPM (app + Nginx)	92,7 ± 0,1	79,2 ± 0,0	80,6
B – CI4 + FrankenPHP clássico	100,2 ± 0,4	123,9 ± 0,2	124,9
E – CI4 + FrankenPHP clássico (2 *threads*, padrão)	98,0 ± 1,7	119,7 ± 0,4	120,8
C – CI4 + FrankenPHP Worker	100,4 ± 0,4	125,4 ± 0,5	127,4
D – Laravel Octane	100,6 ± 0,0	171,6 ± 0,4	172,9

Fonte: elaborado pelo próprio autor (2026), a partir de amostras de `docker stats`. Nota: no cenário A, soma de PHP-FPM e Nginx.

Figura 3 – CPU e RAM médias por cenário



Fonte: elaborado pelo próprio autor (2026).

Em todos os cenários a CPU ficou próxima do limite de 1,0 núcleo (92,7% a 100,6%), confirmando a saturação e que o gargalo do experimento é a CPU, e não a memória: nenhum cenário passou de 173 MiB, contra o limite de 512 MiB. Os cenários persistentes (C e D) usaram a mesma CPU que B e processaram mais requisições; ou seja, converteram o recurso em trabalho útil com mais eficiência, e não consumiram mais por requisição. O cenário A foi o único abaixo de 100%, o que pode refletir a divisão da cota entre PHP-FPM (0,85 CPU) e Nginx (0,15 CPU): como cada contêiner tem seu próprio limite, a capacidade ociosa de um não pode ser usada pelo outro. O PHP-FPM foi o mais econômico em memória (79 MiB), custo esperado de manter a aplicação carregada em memória nos demais: o CodeIgniter 4 em modo Worker usou 125 MiB (+46 MiB em relação ao A) e o Laravel Octane, 172 MiB. Esse acréscimo de memória do modo persistente também foi observado por Dhaniswara, Kusumasari e Alam (2025): em um *endpoint* leve, o FrankenPHP com Laravel Octane usou cerca de 155 MB, contra cerca de 69 MB do PHP-FPM com Nginx, o que os autores interpretam como um *trade-off* entre capacidade de resposta e eficiência de memória.

4.5 Estabilidade da memória nos testes longos

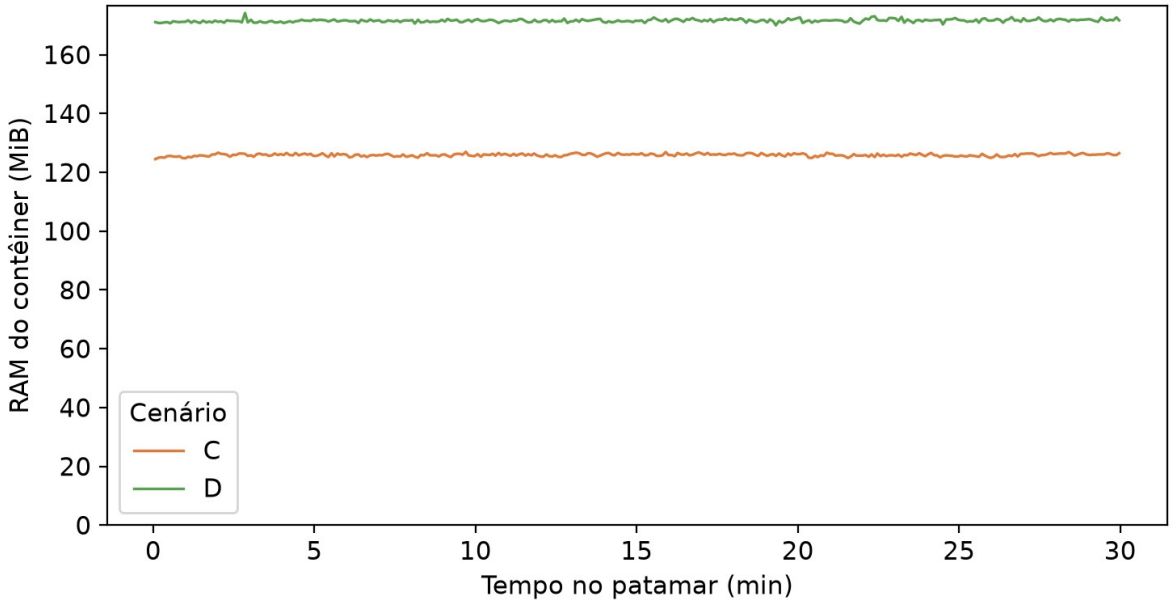
Para verificar se a memória dos cenários persistentes cresce com o tempo, os cenários C e D foram executados com patamar de 30 minutos. A Tabela 5 e a Figura 4 mostram o resultado.

Tabela 5 – Testes longos (patamar de 30 minutos) nos cenários C e D

Cenário	RPS	p95 (ms)	p99 (ms)	RAM nos 5 min iniciais (MiB)	RAM nos 5 min finais (MiB)	RAM pico (MiB)
C – CI4 + FrankenPHP Worker	304,4	397,5	407,6	125,8	126,0	127,0
D – Laravel Octane	340,6	380,0	398,2	171,3	171,8	174,2

Fonte: elaborado pelo próprio autor (2026), a partir dos testes longos e de `docker stats`.

Figura 4 – Memória dos contêineres C e D ao longo do patamar de 30 minutos



Fonte: elaborado pelo próprio autor (2026).

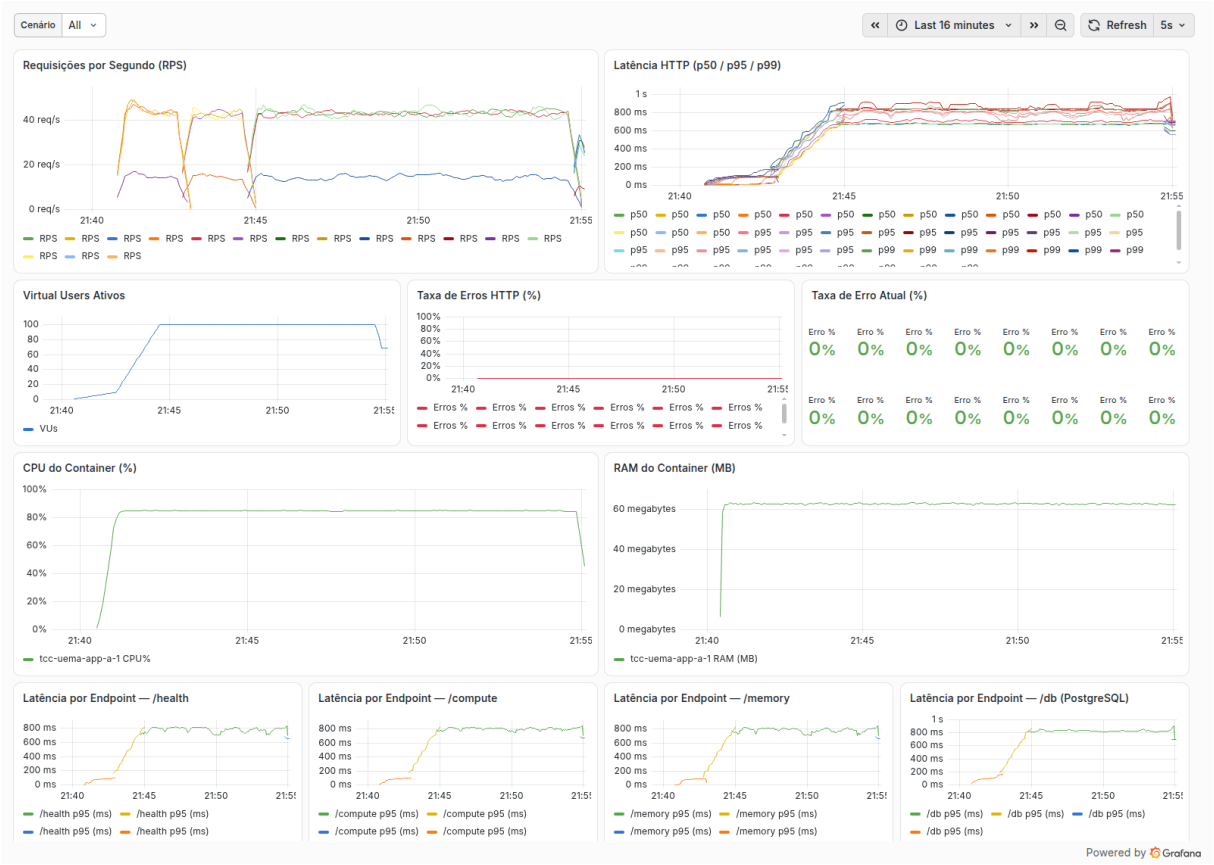
Nos dois cenários a memória se manteve estável durante os 30 minutos, com variação de menos de 1 MiB entre o início e o fim do patamar, e a vazão e a latência ficaram no mesmo nível das execuções de 15 minutos (vazão 0,2% maior no C e 1,1% maior no D). Portanto, não houve evidência de acúmulo de memória nem de degradação de desempenho no modo Worker nem no CodeIgniter 4 nem no Laravel Octane, o que é relevante para a adoção do modo Worker em produção, em que o risco de vazamento entre requisições é a principal contrapartida (DUNGLAS, 2024). A estabilidade em 30 minutos, com 4 *workers*, não garante o mesmo comportamento em períodos de horas ou com aplicações mais complexas.

4.6 Visão temporal nas execuções de captura

As Figuras 5 a 8 mostram o painel do Grafana, e as Figuras 9 a 12 o painel de resumo do k6, de cada cenário em uma execução de captura. Elas ilustram o comportamento no tempo: rampa de usuários virtuais, patamar estável de vazão e latência, CPU constante no teto e erro

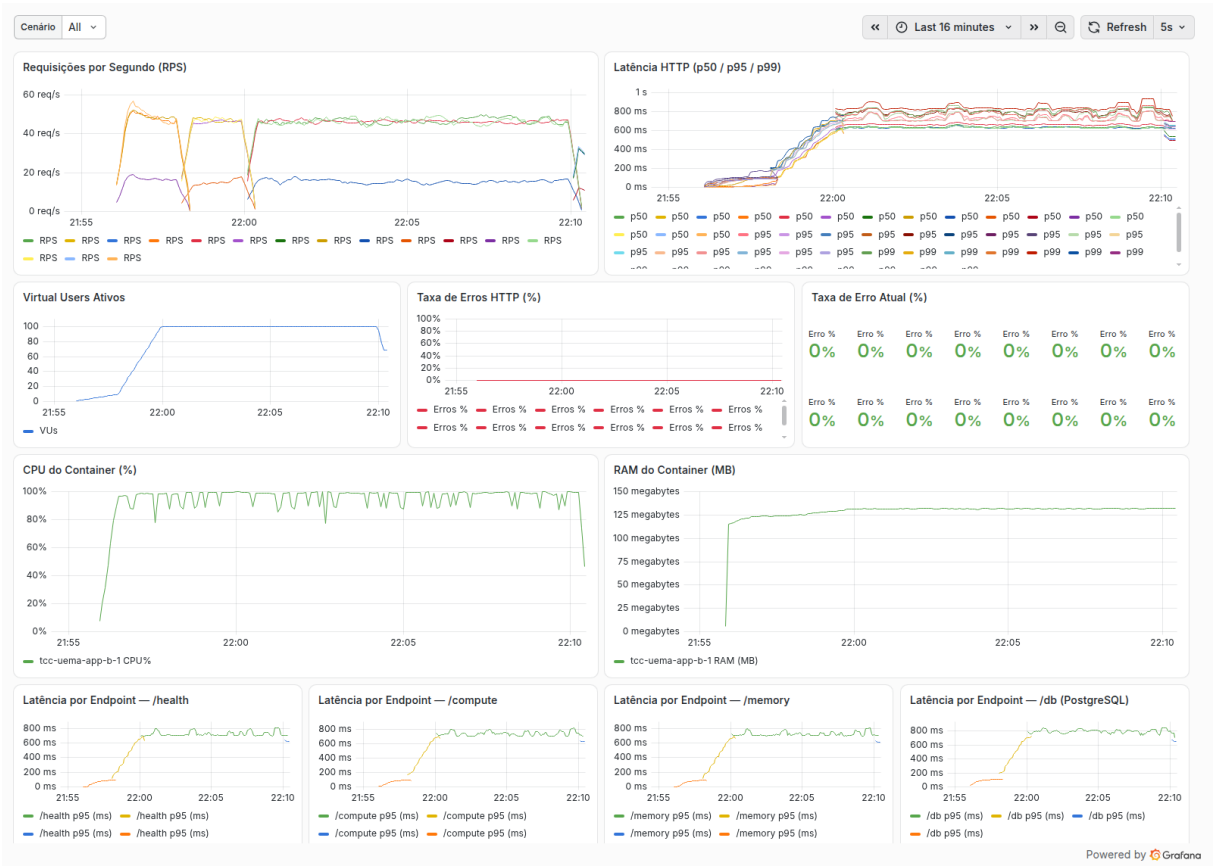
em 0%. No painel do k6, a vazão alta do início da execução corresponde à fase de poucos usuários virtuais, em que as requisições são atendidas sem fila, e cai ao patamar sustentado quando a carga chega a 100 usuários.

Figura 5 – Painel do Grafana, cenário A (execução de captura)



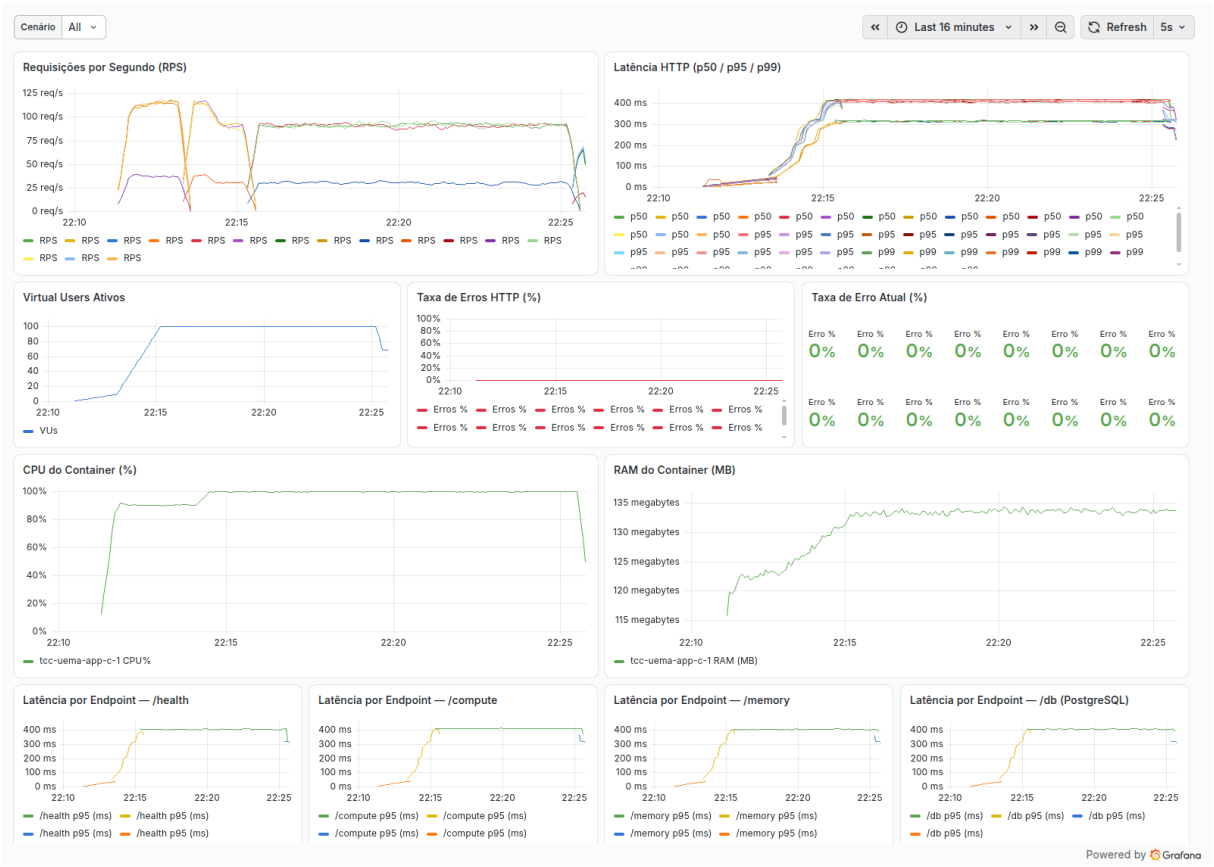
Fonte: elaborado pelo próprio autor (2026).

Figura 6 – Painel do Grafana, cenário B (execução de captura)



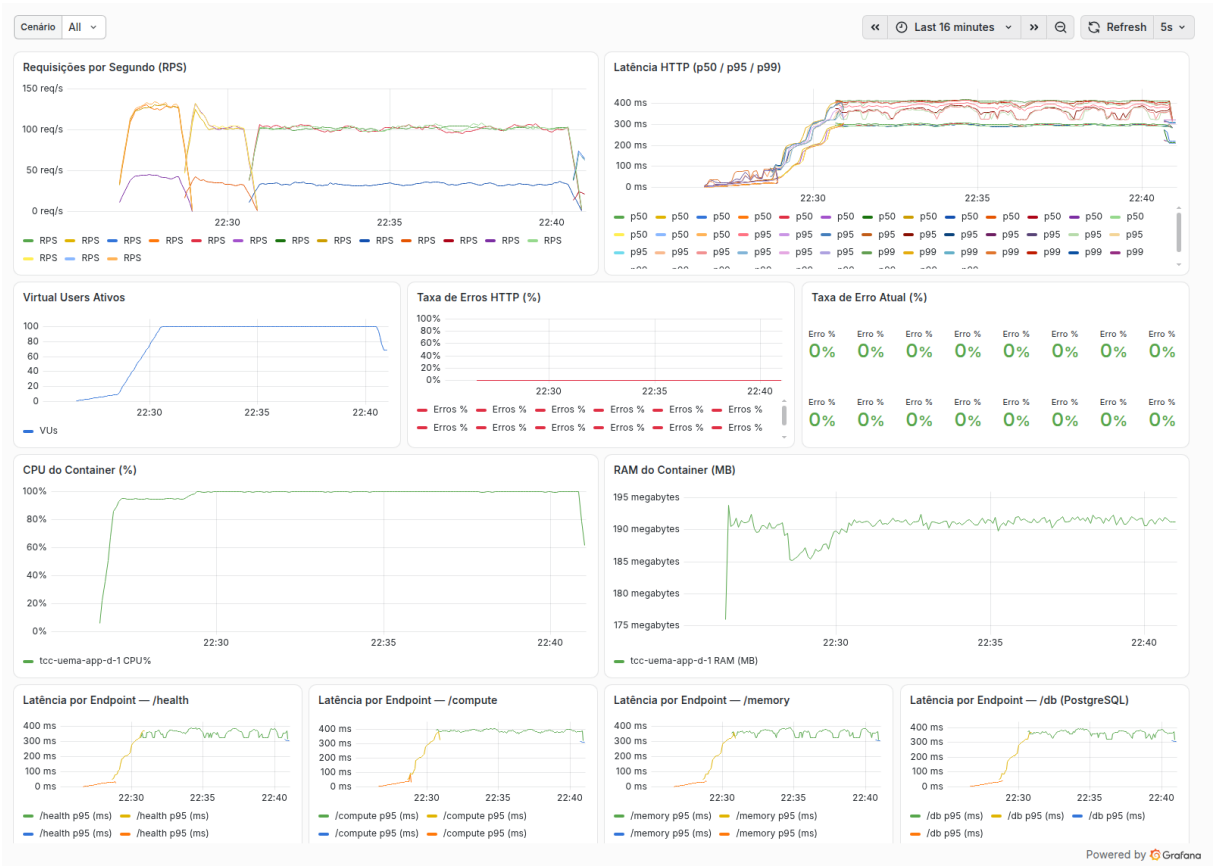
Fonte: elaborado pelo próprio autor (2026).

Figura 7 – Painel do Grafana, cenário C (execução de captura)



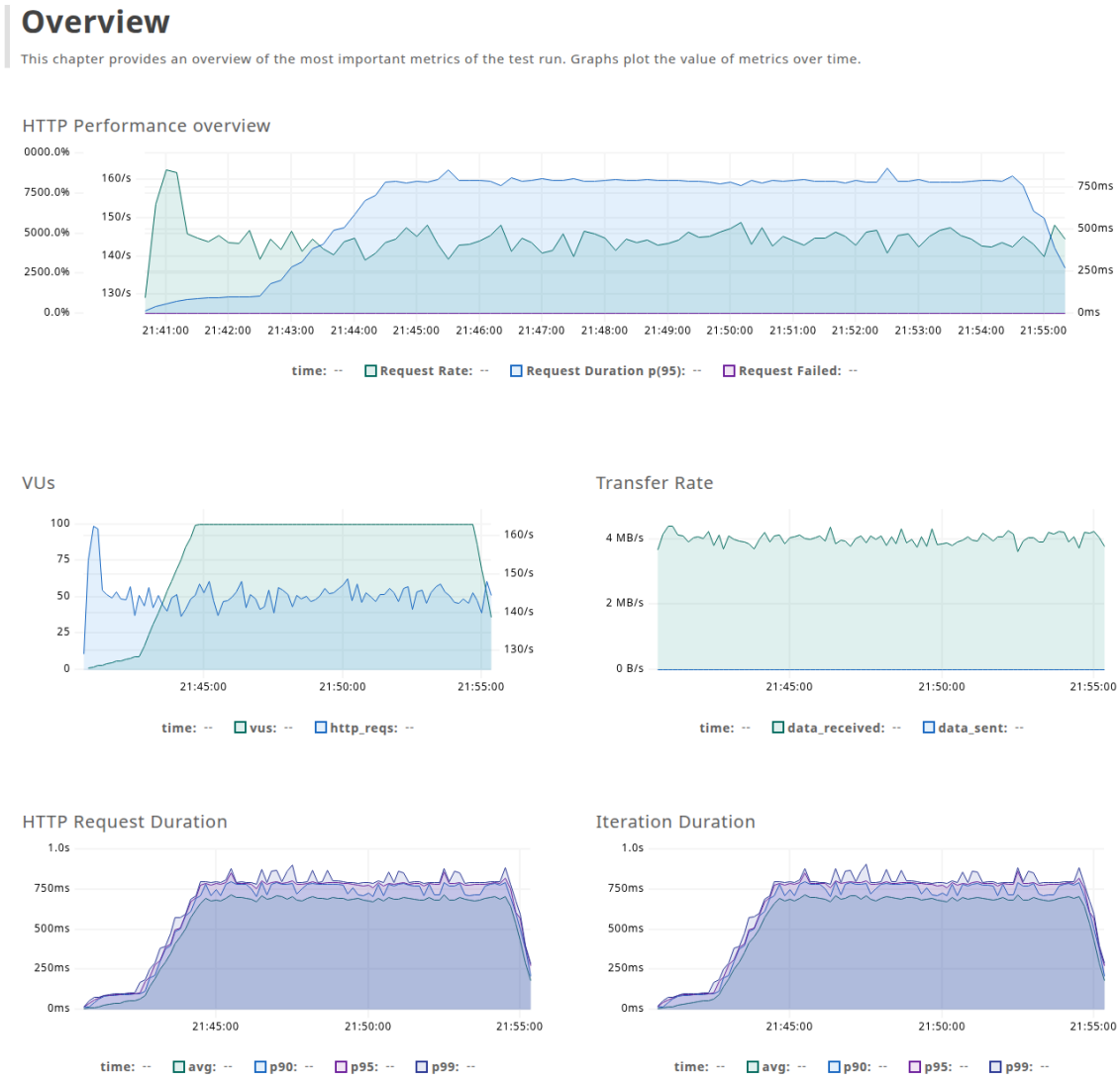
Fonte: elaborado pelo próprio autor (2026).

Figura 8 – Painel do Grafana, cenário D (execução de captura)



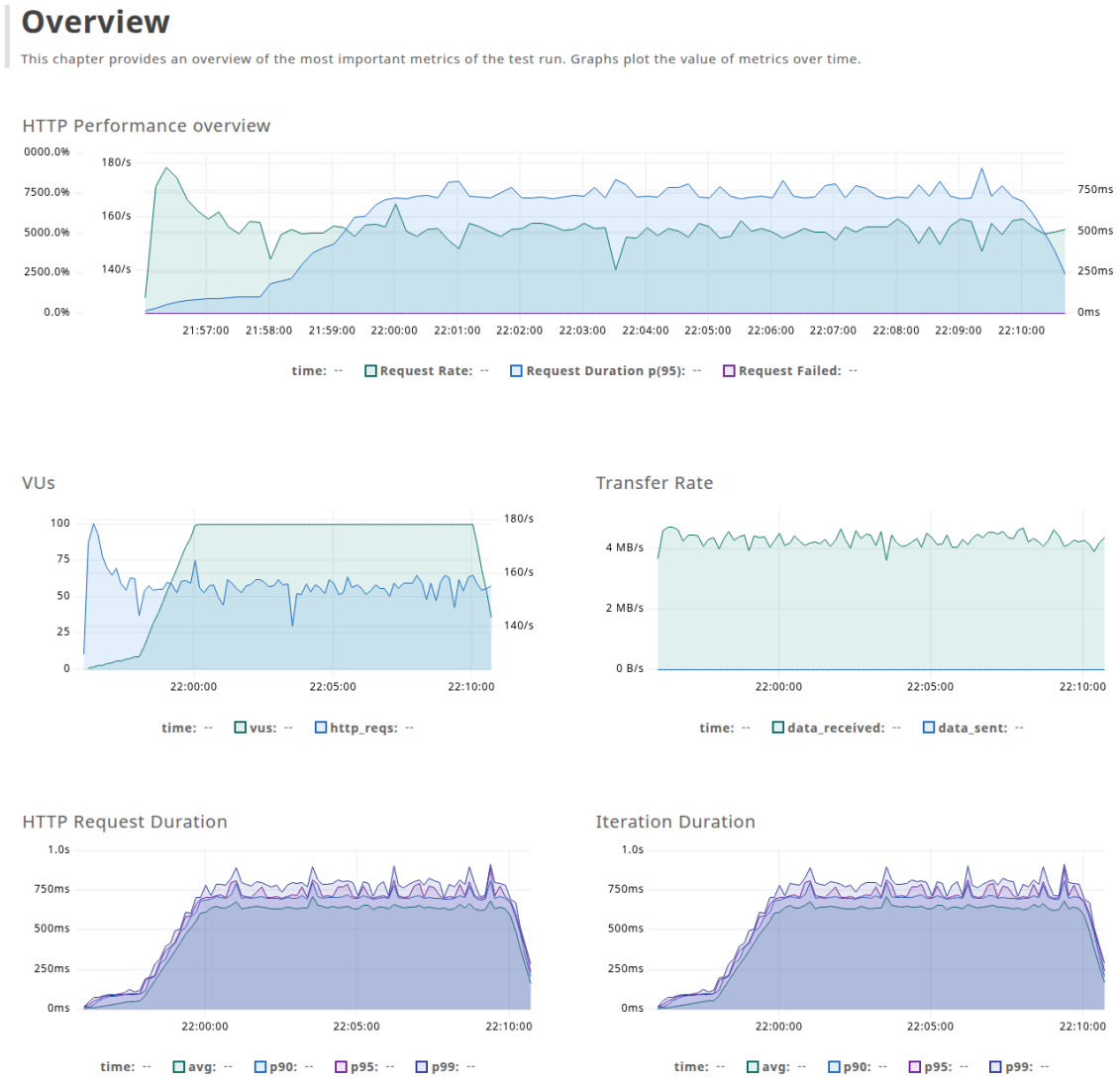
Fonte: elaborado pelo próprio autor (2026).

Figura 9 – Painel do k6, cenário A (execução de captura)



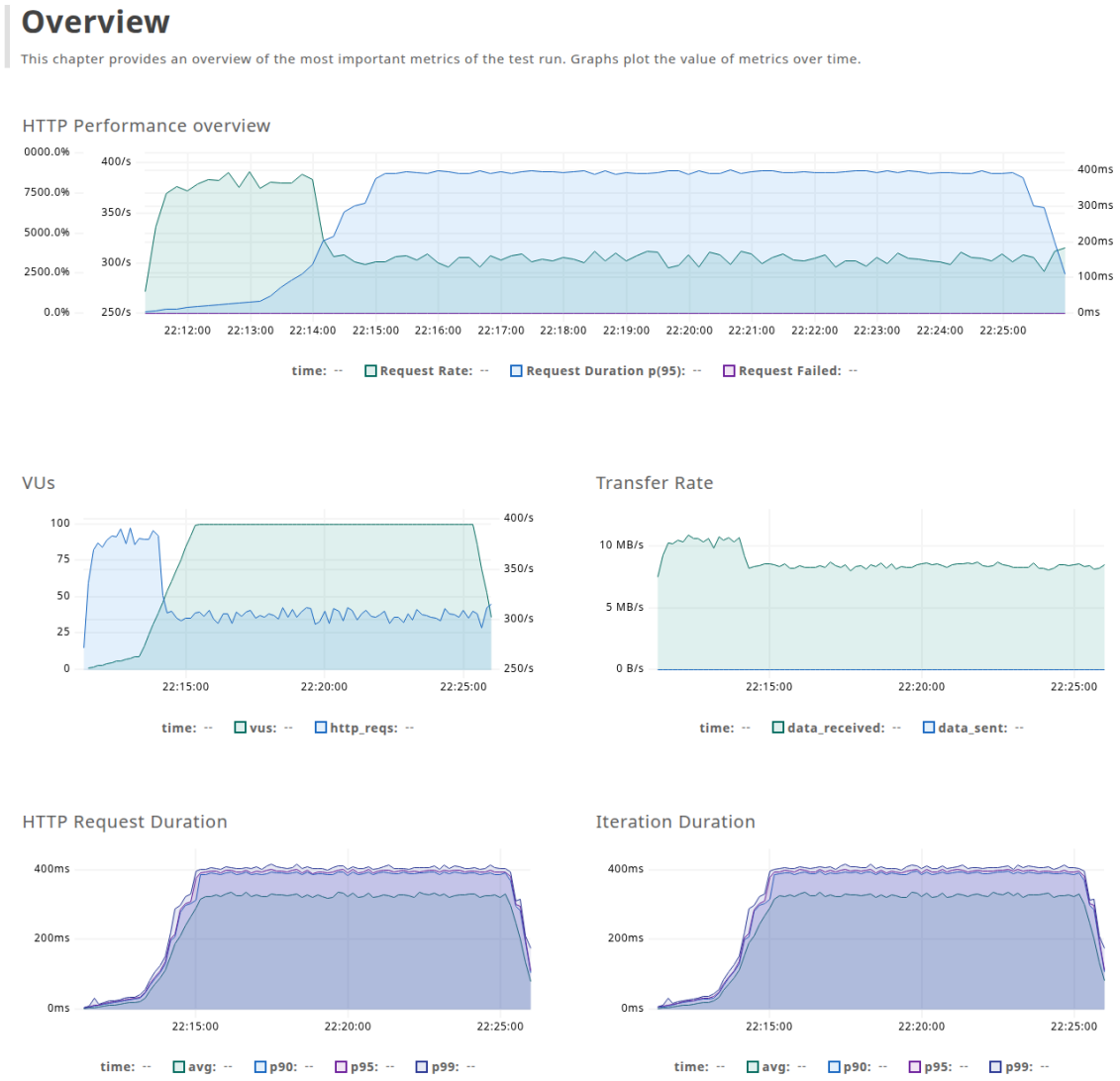
Fonte: elaborado pelo próprio autor (2026).

Figura 10 – Painel do k6, cenário B (execução de captura)



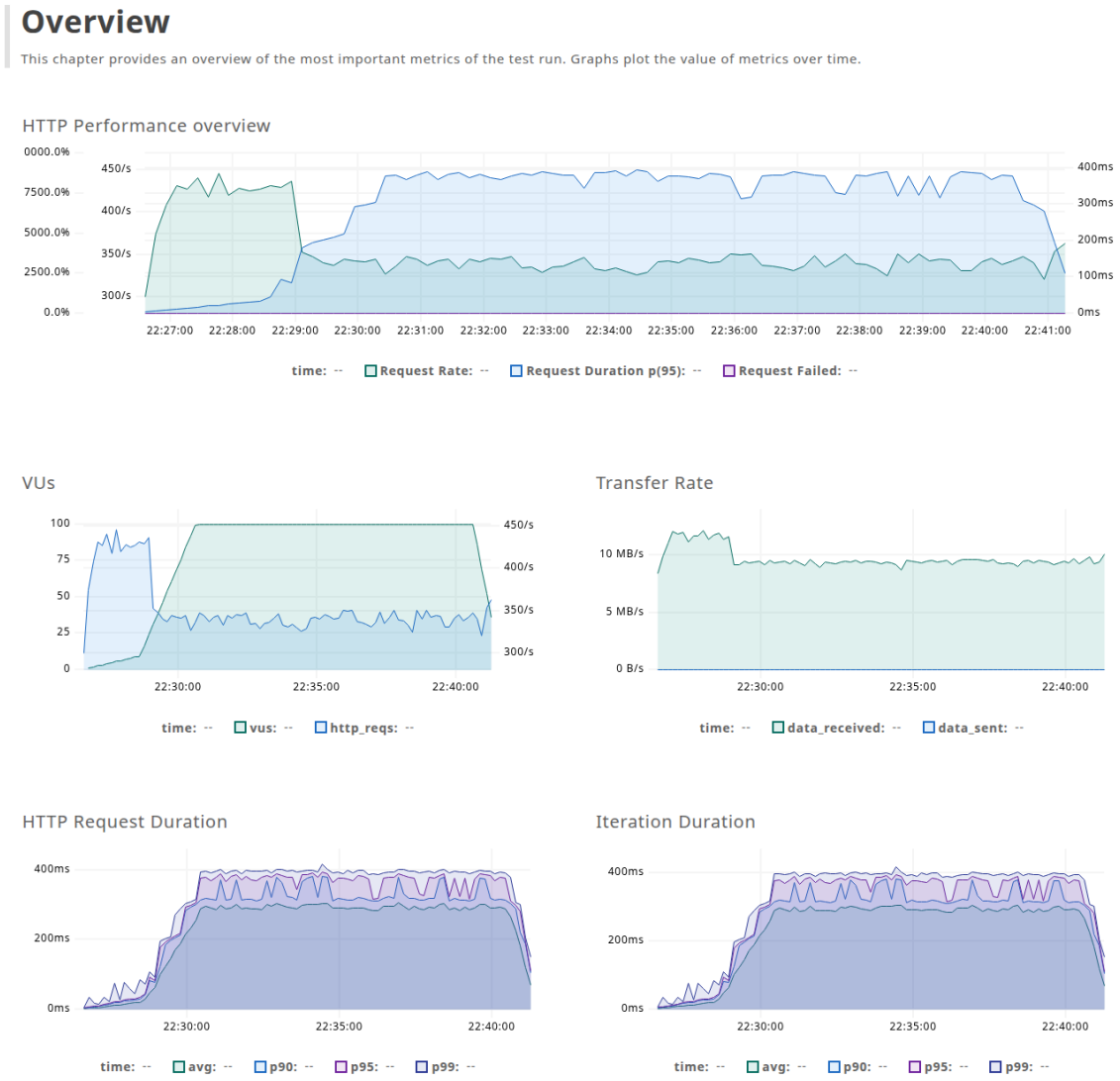
Fonte: elaborado pelo próprio autor (2026).

Figura 11 – Painel do k6, cenário C (execução de captura)



Fonte: elaborado pelo próprio autor (2026).

Figura 12 – Painel do k6, cenário D (execução de captura)



Fonte: elaborado pelo próprio autor (2026).

4.7 Discussão

Persistência do processo reduz o custo de inicialização. O resultado mais forte do estudo é que manter o processo e o *framework* carregados em memória mais que dobrou a vazão do CodeIgniter 4 (C em relação a A, +112,2%) e reduziu pela metade o p95 e o p99, com os mesmos recursos e sem perda de estabilidade (Seção 4.5). Essa é a comparação entre duas arquiteturas completas (servidor, compilação do PHP e divisão de recursos diferentes); a comparação que mais isola o modo Worker é a de C com B, que usam o mesmo FrankenPHP e quatro unidades de concorrência (+95,0% de vazão). Isso é coerente com a motivação do trabalho: no modelo tradicional, a inicialização do *framework* se repete a cada requisição, e o modo Worker do FrankenPHP elimina essa repetição ao manter o processo vivo (DUNGLAS,

2024). O Quadro 1 mostra que, no CodeIgniter 4, o modo Worker preserva também a conexão com o banco e o estado da aplicação, de modo que o ganho decorre da soma da eliminação do *bootstrap* com o reaproveitamento da conexão; o experimento não separa as duas parcelas. Resultado semelhante em direção, ainda que em outro contexto, é relatado por Luz, Silva e Emer (2024), que encontraram vantagem estatisticamente significativa do modelo assíncrono do Swoole sobre o PHP síncrono tradicional em testes de carga, e por Azzahidi, Wijayanto e Darmawan (2025), que incluíram Laravel com FrankenPHP entre os *frameworks* avaliados e o destacaram pelo uso eficiente de memória. Nossos resultados complementam esses estudos ao isolar o efeito do servidor (B) do efeito do modo Worker (C) em um mesmo *framework*.

Trocar apenas o servidor traz um ganho pequeno. O FrankenPHP em modo clássico (B) superou o PHP-FPM (A) em 8,8% de vazão, mas não de modo significativo no p95 (−5,9%) nem no p99 (−1,7%), e ficou 49% abaixo do cenário C. O cenário B é a comparação mais direta com o A, pois ambos reiniciam estado e conexão a cada requisição (Quadro 1). Sem o cenário B, o ganho do cenário C poderia ser atribuído erroneamente ao FrankenPHP em si, quando decorre sobretudo do modo Worker. Neste experimento não investigamos a causa do pequeno ganho de B sobre A; uma hipótese é que o FrankenPHP dispensa o salto FastCGI entre Nginx e PHP-FPM e a divisão da cota de CPU entre dois contêineres (Seção 4.4), compensada em parte pelo uso de PHP *thread-safe*. O cenário de sensibilidade E, com o número padrão de *threads* do servidor, elevou a vazão em 13,2% em relação ao B, o que mostra que o resultado do modo clássico depende dessa configuração. Com o limite de 1,0 CPU, esse padrão equivale a 2 *threads* PHP, menos que as 4 do B; ou seja, o cenário com menos *threads* obteve mais vazão. Uma hipótese, não investigada neste trabalho, é que, com a cota de uma CPU já saturada, menos *threads* concorrentes reduzam a disputa pelo processador e a troca de contexto; o experimento não separa essa explicação de outras. Mesmo assim, E ficou 42% abaixo do cenário C em vazão. A conclusão de que o modo Worker é o que determina o ganho vale, portanto, para as duas configurações do modo clássico.

O CodeIgniter 4 em Worker ficou próximo do Laravel Octane, atrás dele. O Laravel Octane (D) superou o CodeIgniter 4 em modo Worker (C) em 10,9% de vazão (336,9 contra 303,9 RPS) e 3,6% de p95, diferença menor que a que separa qualquer cenário clássico do modo Worker, mas significativa na vazão mesmo após a correção de Bonferroni (Tabela 2), e relativa a duas implementações específicas, sob uma carga e uma configuração particulares. O resultado diverge da expectativa implícita no problema de pesquisa, de que o CodeIgniter, por ser mais leve, aproveitaria melhor o modo Worker: ele ficou 9,8% abaixo do Octane, e com menos memória (125 contra 172 MiB, Tabela 4). Duas observações ajudam a interpretar: (i) o

Octane é um pacote maduro, projetado para esse modelo de execução e que já trata a limpeza de estado entre requisições (OTWELL, 2024), enquanto o suporte nativo do CodeIgniter ao modo Worker é recente (CODEIGNITER FOUNDATION, 2024); e (ii) o diagnóstico mostrou que os dois preservam estado e conexão, de modo que a diferença remanescente não decorre da persistência da conexão. Não foi feito perfilamento do código, de modo que as causas permanecem hipóteses; o que os dados sustentam é a diferença de vazão, o comportamento da memória e a equivalência de estado preservado.

Comparação com o estudo mais próximo na literatura. Dhaniswara, Kusumasari e Alam (2025) compararam PHP-FPM com Nginx e FrankenPHP em modo Worker com Laravel Octane e, em um *endpoint* de carga leve, observaram vantagem do FrankenPHP em tempo de resposta médio (−15,8%), vazão (+18,6%) e latência de cauda (p99 de 574 contra 794 ms), com maior uso de memória (cerca de 155 contra 69 MB). Há concordância na direção dos resultados: o modo Worker melhorou a vazão e a latência, inclusive na cauda, ao custo de mais memória, e em ambos os estudos a CPU foi o recurso saturado. A magnitude, porém, difere: aqui o CodeIgniter 4 em modo Worker teve vazão 112,2% maior que a do PHP-FPM e p99 49,4% menor, ganhos bem superiores aos relatados por eles. Não é possível atribuir a diferença a uma única causa, pois os estudos diferem em *framework* (Laravel contra CodeIgniter 4), *endpoints*, banco de dados (MySQL contra PostgreSQL), ferramenta de carga (JMeter contra k6), versão do PHP (8.2 contra 8.3), recursos (4 núcleos contra 1,0 CPU) e arquitetura comparada (eles confrontam o modo Worker diretamente com o PHP-FPM, sem o modo clássico do FrankenPHP como cenário intermediário, que aqui trouxe ganho pequeno). A comparação deve ser lida, portanto, como convergência qualitativa, e não como replicação. Os autores observaram ainda que, em um *endpoint* que serializa grande volume de dados, as duas arquiteturas saturaram a CPU com desempenho semelhante, com gargalo na aplicação; os *endpoints* deste estudo são simples e não exercitam essa condição (Seção 4.8).

Quanto a reprodutibilidade e benchmarking. A decisão de repetir cada cenário três vezes, em ordem rotativa, de manter a mesma base de sistema operacional em todos eles e de reportar o desvio-padrão segue a recomendação da literatura de que benchmarks de software sofrem instabilidade e devem ser tratados com repetição e controle do ambiente (LAABER; BASMACI; SALZA, 2021), e a de que comparações de desempenho exigem condições idênticas de execução (ZHAN, 2022; BRUNST et al., 2022; JIA et al., 2013). O desvio-padrão baixo observado (no máximo 1,0% da média) indica que o ambiente conteinerizado, com limites estritos de CPU e memória, produziu medições estáveis.

4.8 Ameaças à validade e limitações

Os resultados devem ser lidos considerando as seguintes limitações do estudo:

a) **hospedeiro único e injetor no mesmo hospedeiro:** o k6 rodou na mesma máquina que os servidores (em contêiner com 2,0 CPU), o que pode causar interferência, embora igual entre cenários. Outros processos do computador do autor consumiam memória durante as medições;

b) **ordem das execuções:** os cenários A a D foram executados em ordem rotativa, mas o cenário E foi executado depois deles, e os testes longos por último; a temperatura do hospedeiro pode ter variado entre esses blocos;

c) **carga sintética e modelo fechado:** os endpoints são simples e a carga é de usuários virtuais sem tempo de espera, o que satura o servidor e torna a latência função da vazão (Seção 4.3). Os resultados não se estendem automaticamente a aplicações reais, com consultas mais complexas e uso de sessão; em um *endpoint* que serializa grande volume de dados, Dhaniswara, Kusumasari e Alam (2025) observaram que o modo Worker e o PHP-FPM tiveram desempenho semelhante, com o gargalo na aplicação e não no servidor, o que indica que o ganho depende da carga;

d) **tamanho da amostra:** com três execuções por cenário, os testes estatísticos têm poder limitado e a normalidade não pode ser verificada; as diferenças de p95 e p99 de B sobre A e de E sobre B não são significativas;

e) **carga única:** a vazão foi medida apenas com 100 usuários virtuais, sem varredura de concorrência; por isso é a vazão observada nessa carga, e não se demonstra que seja o máximo de cada cenário;

f) **compilação do PHP:** o PHP do FrankenPHP é *thread-safe* e o do PHP-FPM não é, diferença inerente às arquiteturas que o experimento não separa do efeito do servidor;

g) **infraestrutura de medição não fixada:** as imagens do Prometheus, do Grafana, do cAdvisor e do k6 usam a etiqueta `latest`, ao contrário das imagens dos cenários, fixadas por versão e por *digest*. Elas são idênticas entre cenários, mas não garantem a reprodução bit a bit;

h) **causas não investigadas:** diferenças entre A e B e entre C e D foram interpretadas como hipóteses, sem perfilamento.

5 CONSIDERAÇÕES FINAIS

Este trabalho investigou se a arquitetura de execução persistente em memória justifica a complexidade de adoção no *framework* CodeIgniter 4, comparando o PHP-FPM tradicional (cenário A), o FrankenPHP em modo clássico (cenário B) e o FrankenPHP em modo Worker (cenário C), com o Laravel Octane (cenário D) como referência de mercado. O objetivo geral, comparar o desempenho do CodeIgniter 4 sob essas arquiteturas com o mesmo orçamento de recursos, foi atingido. Além dos quatro cenários previstos, foram incluídos, como complementos, um cenário de sensibilidade (E, FrankenPHP clássico com o número padrão de *threads*), testes longos de 30 minutos, o percentil 99 e testes estatísticos (ANOVA e testes t de Welch).

Quanto ao primeiro objetivo específico, montar um ambiente experimental conteinerizado e reprodutível, com limites idênticos de CPU e memória, o experimento foi concluído com Docker Compose, limites estritos de recursos, imagens de aplicação e servidor web com a mesma base (Debian 13) e as mesmas versões de PHP, k6 como injetor e Prometheus e Grafana na observação. Ele produziu medições estáveis, com desvio-padrão de no máximo 1,0% da média em três execuções por cenário e 0,00% de erros.

Quanto ao segundo, medir e comparar vazão e latência, os resultados mostram que o modo Worker do FrankenPHP elevou a vazão do CodeIgniter 4 em 112,2% em relação ao PHP-FPM (303,9 contra 143,2 RPS, no patamar de 10 minutos sob 100 usuários virtuais) e reduziu o p95 de 789,7 para 397,9 ms e o p99 de 807,2 para 408,3 ms; em relação ao FrankenPHP clássico, que usa o mesmo servidor, o ganho de vazão foi de 95,0%. O FrankenPHP em modo clássico entregou ganho modesto sobre o PHP-FPM (+8,8% de vazão com 4 *threads* e +23,2% com o número padrão de *threads*, que nas condições do experimento são 2), muito inferior ao do modo Worker. O Laravel Octane obteve a maior vazão (336,9 RPS), 10,9% acima do CodeIgniter 4 em modo Worker, diferença pequena que, ainda assim, resistiu à correção de Bonferroni (p ajustado = 0,001), mas é relativa a duas implementações específicas.

Quanto ao terceiro, avaliar o consumo de CPU e memória, todos os cenários saturaram a CPU (92,7% a 100,6% de um núcleo) e nenhum se aproximou do limite de memória (no máximo 173 MiB, de 512 MiB). O custo do modo Worker foi um uso maior de RAM: 125 MiB no CodeIgniter 4 e 172 MiB no Laravel Octane, contra 79 MiB do PHP-FPM. Nos testes longos de 30 minutos, a memória dos dois cenários persistentes permaneceu estável, com variação inferior a 1 MiB, sem evidência de acúmulo entre requisições.

Quanto ao quarto, verificar o que é preservado entre requisições, constatou-se que o CodeIgniter 4 em modo Worker preserva o estado da aplicação (o contador do endpoint `/memory` cresce em cada *worker*) e a conexão com o banco (um PID de conexão por *worker*), exatamente como o Laravel Octane, ao passo que PHP-FPM e FrankenPHP clássico reiniciam ambos a cada requisição. O Octane diferencia-se por reciclar periodicamente os *workers*, o que o CodeIgniter 4 não faz.

Quanto ao quinto, discutir se o ganho justifica a complexidade de adoção, os dados indicam que sim, nas condições testadas: mais que dobrar a vazão e reduzir à metade a latência de cauda com os mesmos recursos, ao custo de 46 MiB adicionais de memória, sem instabilidade em 30 minutos e com suporte nativo do *framework* (gerado por um único comando, `php spark worker:install`). A troca isolada do servidor, mantendo o modo clássico, não se justificaria apenas pelo desempenho. A contrapartida permanece sendo a responsabilidade de evitar o vazamento de estado entre requisições, que o experimento, com *endpoints* simples, não exercitou a fundo.

Respondendo à pergunta de pesquisa, a execução persistente em memória do FrankenPHP elevou a vazão, reduziu a latência e aumentou o consumo de memória do CodeIgniter 4, aproximando-o do Laravel Octane. Como contribuições, o estudo oferece dados empíricos reprodutíveis, com ambiente totalmente automatizado, sobre o suporte nativo do CodeIgniter 4 ao modo Worker (introduzido na versão 4.7), ainda pouco documentado na literatura, e separa o efeito do servidor do efeito do modo de execução em um mesmo *framework*. Os resultados concordam, na direção, com o estudo de Dhaniswara, Kusumasari e Alam (2025) com Laravel, em que o FrankenPHP em modo Worker também melhorou vazão e latência ao custo de mais memória, embora com magnitude diferente. Para equipes que mantêm aplicações em CodeIgniter 4, os resultados indicam que vale avaliar o modo Worker.

Entre as limitações, destacam-se: execução em um único hospedeiro, com o injetor de carga na mesma máquina; carga sintética em modelo fechado, que satura o servidor e faz a latência refletir a vazão, e com uma única concorrência (100 usuários virtuais), de modo que se mede a vazão observada nessa carga e não a capacidade máxima de cada cenário; apenas três execuções por cenário, o que limita o poder estatístico; diferença de compilação do PHP (*thread-safe* no FrankenPHP e não no PHP-FPM), inerente às arquiteturas; e causas das diferenças (A contra B e C contra D) tratadas como hipóteses, sem perfilamento do código.

Como trabalhos futuros, sugere-se: repetir o experimento com carga de taxa de chegada fixa (modelo aberto), para isolar o custo de cada *endpoint* e do reaproveitamento da conexão; ampliar o número de execuções por cenário; realizar testes de duração de várias horas e com

aplicações mais realistas, com consultas, sessões e outros serviços, para investigar o vazamento de estado; perfilar o código para explicar a diferença entre o CodeIgniter 4 em Worker e o Laravel Octane e entre o PHP-FPM e o FrankenPHP clássico; testar *endpoints* com grande volume de dados, que no estudo de Dhaniswara, Kusumasari e Alam (2025) levaram as duas arquiteturas ao limite de CPU, para verificar até que ponto o ganho do modo Worker depende da carga; e comparar outras alternativas persistentes, como Swoole e RoadRunner.

REFERÊNCIAS

AZZAHIDI, Aufa Syaihan; WIJAYANTO, Bangun; DARMAWAN, Agus. Performance Evaluation of Backend Frameworks for REST API: A Comparative Study of Spring Boot, Flask, Express.js, Laravel FrankenPHP, and Gin. **Jurnal Teknik Informatika (JUTIF)**, v. 6, n. 4, p. 2405-2419, ago. 2025. DOI: https://doi.org/10.52436/1.jutif.2025.6.4.4811.

BRUNST, Holger et al. **First Experiences in Performance Benchmarking with the New SPEChpc 2021 Suites**. 2022. Disponível em: https://arxiv.org/abs/2203.06751. Acesso em: 19 out. 2026.

CODEIGNITER FOUNDATION. **CodeIgniter 4 User Guide**. 2024. Disponível em: https://codeigniter.com/user_guide/. Acesso em: 19 out. 2026.

DHANISWARA, Abiyoga; KUSUMASARI, Tien Fabrianti; ALAM, Ekky Novriza. Performance Evaluation of Laravel Application Deployment on Docker. In: INTERNATIONAL CONFERENCE ON INFORMATION TECHNOLOGY AND COMPUTING (ICITCOM), 2025, [S. l.]. **Proceedings** [...]. [S. l.]: IEEE, 2025. p. 170-175. DOI: https://doi.org/10.1109/ICITCOM66635.2025.11265204.

DUNGLAS, Kévin. **FrankenPHP: The Modern Application Server for PHP**. LesTilleuls.coop, 2024. Disponível em: https://frankenphp.dev/docs/. Acesso em: 19 out. 2026.

GRAFANA LABS. **k6 Documentation: Load Testing for Engineering Teams**. 2024. Disponível em: https://grafana.com/docs/k6/latest/using-k6/metrics/. Acesso em: 19 out. 2026.

JIA, Zhen et al. **The Implications of Diverse Applications and Scalable Data Sets in Benchmarking Big Data Systems**. 2013. Disponível em: https://arxiv.org/abs/1307.7943. Acesso em: 19 out. 2026.

LAABER, Christoph; BASMACI, Mikael; SALZA, Pasquale. Predicting unstable software benchmarks using static source code features. **Empirical Software Engineering**, v. 26, art. 114, 2021. DOI: https://doi.org/10.1007/s10664-021-09996-y.

LITTLE, John D. C. A Proof for the Queuing Formula: L = λW. **Operations Research**, v. 9, n. 3, p. 383-387, 1961. DOI: https://doi.org/10.1287/opre.9.3.383.

LUZ, Jailson Maciel da; SILVA, Roger Sá da; EMER, Cassandro Davi. **Otimização de Aplicações Web com Programação Assíncrona em PHP e Swoole**. Veranópolis: Instituto Federal de Educação, Ciência e Tecnologia do Rio Grande do Sul (IFRS), Campus Avançado Veranópolis, 2024.

OTWELL, Taylor. **Laravel Octane Documentation**. Laravel LLC, 2024. Disponível em: https://laravel.com/docs/13.x/octane. Acesso em: 19 out. 2026.

ZHAN, Jianfeng et al. **A BenchCouncil View on Benchmarking Emerging and Future Computing**. 2022. Disponível em: https://arxiv.org/abs/2205.07769. Acesso em: 19 out. 2026.

APÊNDICE A – ORQUESTRAÇÃO DOS CONTÊINERES (DOCKER-COMPOSE.YML)

docker-compose.yml

```
services:

  # ==============================================================================
  # CENÁRIO A — Legacy: CodeIgniter 4 + PHP-FPM + Nginx (base Debian 13, igual aos
    demais)
  # Topologia real de produção PHP tradicional
  # ==============================================================================
  app-a:
    profiles: [scenario-a]
    build:
      context: .
      dockerfile: docker/scenario-a/Dockerfile
    expose:
      - "9000"
    deploy:
      resources:
        limits:
          # 0.85 CPU / 448MB — orçamento do processo PHP-FPM
          # nginx-a recebe 0.15 CPU / 64MB: juntos totalizam 1.0 CPU / 512MB
          # Isso iguala o orçamento total do Cenário A aos demais (B, C, D)
          cpus: "0.85"
          memory: "448M"
    environment:
      SCENARIO_NAME: scenario-a
      COMPUTE_SIZE: "1000"
      CI_ENVIRONMENT: production
    volumes:
      - ./docker/scenario-a/php-fpm.conf:/usr/local/etc/php-fpm.d/www.conf
    depends_on:
      postgres:
        condition: service_healthy
    networks:
      - bench-net
    restart: unless-stopped

  nginx-a:
    profiles: [scenario-a]
    image:
        nginx:1.31.7-trixie@sha256:bc4cad49969aaf95eb4eecec5572846f86e59f1e8332aba36ed64
        b91aff8b724
    ports:
      - "${BENCH_PORT:-8080}:8080"
    deploy:
      resources:
        limits:
          # 0.15 CPU / 64MB — orçamento do reverse proxy Nginx
          # Nginx é muito leve; esse limite reflete seu papel no stack
          # Total Cenário A: app-a (0.85/448M) + nginx-a (0.15/64M) = 1.0 CPU / 512MB
          cpus: "0.15"
          memory: "64M"
    volumes:
      - ./docker/scenario-a/nginx.conf:/etc/nginx/conf.d/default.conf
      - ./apps/ci4-base/public:/var/www/html/public:ro
    depends_on:
      - app-a
    networks:
      - bench-net
    restart: unless-stopped

  # ==============================================================================
  # CENÁRIO B — Modern Classic: CodeIgniter 4 + FrankenPHP Classic Mode
  # FrankenPHP como substituto do PHP-FPM (sem Worker, sem estado persistente)
  # Isola o ganho do motor FrankenPHP vs o Worker Mode em si
  # ==============================================================================
  app-b:
    profiles: [scenario-b]
    build:
```

```
    context: .
    dockerfile: docker/scenario-b/Dockerfile
  ports:
    - "${BENCH_PORT:-8080}:8080"
  deploy:
    resources:
      limits:
        cpus: "1.0"
        memory: "512M"
  environment:
    SCENARIO_NAME: scenario-b
    COMPUTE_SIZE: "1000"
    CI_ENVIRONMENT: production
    SERVER_NAME: ":8080"
  depends_on:
    postgres:
      condition: service_healthy
  networks:
    - bench-net
  restart: unless-stopped

# Sensibilidade: FrankenPHP clássico com número PADRÃO de threads (sem num_threads)
app-e:
  profiles: [scenario-e]
  build:
    context: .
    dockerfile: docker/scenario-b/Dockerfile
  ports:
    - "${BENCH_PORT:-8080}:8080"
  deploy:
    resources:
      limits:
        cpus: "1.0"
        memory: "512M"
  volumes:
    - ./docker/scenario-b/Caddyfile.default:/etc/frankenphp/Caddyfile:ro
  environment:
    SCENARIO_NAME: scenario-e
    COMPUTE_SIZE: "1000"
    CI_ENVIRONMENT: production
    SERVER_NAME: ":8080"
  depends_on:
    postgres:
      condition: service_healthy
  networks:
    - bench-net
  restart: unless-stopped

# ============================================================================
# CENÁRIO C — Vanguard: CodeIgniter 4 + FrankenPHP Worker Mode
# Bridge customizada mantém o CI4 em memória persistente entre requests
# Este é o cenário central do TCC
# ============================================================================
app-c:
  profiles: [scenario-c]
  build:
    context: .
    dockerfile: docker/scenario-c/Dockerfile
  ports:
    - "${BENCH_PORT:-8080}:8080"
  deploy:
    resources:
      limits:
        cpus: "1.0"
        memory: "512M"
  environment:
    FRANKENPHP_WORKERS: "${FRANKENPHP_WORKERS:-4}"
    SCENARIO_NAME: scenario-c
    COMPUTE_SIZE: "1000"
    CI_ENVIRONMENT: production
    SERVER_NAME: ":8080"
  depends_on:
    postgres:
      condition: service_healthy
  networks:
    - bench-net
```

```
  restart: unless-stopped

# ==============================================================================
# CENÁRIO D — Referência: Laravel + Laravel Octane + FrankenPHP
# Benchmark de mercado — Octane suporta Worker Mode nativamente
# ==============================================================================
app-d:
  profiles: [scenario-d]
  build:
    context: .
    dockerfile: docker/scenario-d/Dockerfile
  command: ["php", "artisan", "octane:frankenphp", "--workers=${OCTANE_WORKERS:-4}",
     "--port=8080", "--host=0.0.0.0"]
  ports:
    - "${BENCH_PORT:-8080}:8080"
  deploy:
    resources:
      limits:
        cpus: "1.0"
        memory: "512M"
  environment:
    SCENARIO_NAME: scenario-d
    COMPUTE_SIZE: "1000"
    APP_ENV: production
    APP_DEBUG: "false"
    OCTANE_SERVER: frankenphp
    SERVER_NAME: ":8080"
  depends_on:
    postgres:
      condition: service_healthy
  networks:
    - bench-net
  restart: unless-stopped

# ==============================================================================
# BANCO DE DADOS — PostgreSQL 16
# Compartilhado entre todos os cenários (roda um de cada vez via profiles)
# Recurso isolado: NÃO conta no orçamento de 1.0 CPU / 512MB dos apps
#
# Por que PostgreSQL para o benchmark:
#   - Modelo process-per-connection (~5-10MB RAM/conexão) torna o custo de
#     abrir uma nova conexão mais caro que MySQL (thread-based)
#   - Amplifica exatamente o diferencial do Worker Mode (conexão persistente)
#   - Cenário A (PHP-FPM): nova conexão PG por request = overhead mensurável
#   - Cenário C (Worker Mode): conexão reutilizada = sem overhead de handshake
# ==============================================================================
postgres:
  profiles: [scenario-a, scenario-b, scenario-c, scenario-d, scenario-e]
  image: postgres:16-alpine
  environment:
    POSTGRES_DB:       benchmark
    POSTGRES_USER:     benchmark
    POSTGRES_PASSWORD: benchmark
  command: >
    postgres
      -c max_connections=20
      -c shared_buffers=64MB
      -c log_min_messages=WARNING
  volumes:
    - ./docker/postgres/init.sql:/docker-entrypoint-initdb.d/init.sql:ro
    - postgres-data:/var/lib/postgresql/data
  healthcheck:
    test: ["CMD-SHELL", "pg_isready -U benchmark -d benchmark"]
    interval: 5s
    timeout: 3s
    retries: 10
    start_period: 10s
  deploy:
    resources:
      limits:
        cpus: "0.5"
        memory: "256M"
  networks:
    - bench-net
  restart: unless-stopped
```

```
# ============================================================================
# INJETOR DE CARGA — Grafana k6
# 2.0 CPU / 512MB RAM conforme especificado no TCC
# Executa apenas via `docker compose run --rm k6 run /scripts/...`
# ============================================================================
k6:
  profiles: [scenario-a, scenario-b, scenario-c, scenario-d, scenario-e]
  image: grafana/k6:latest
  user: root
  volumes:
    - ./k6:/scripts
    - ./results:/results
  environment:
    K6_PROMETHEUS_RW_SERVER_URL: http://prometheus:9090/api/v1/write
    K6_PROMETHEUS_RW_TREND_AS_NATIVE_HISTOGRAM: "true"
  deploy:
    resources:
      limits:
        cpus: "2.0"
        memory: "512M"
  networks:
    - bench-net
  depends_on:
    - prometheus

# ============================================================================
# OBSERVABILIDADE
# ============================================================================

prometheus:
  profiles: [scenario-a, scenario-b, scenario-c, scenario-d, scenario-e]
  image: prom/prometheus:latest
  ports:
    - "9090:9090"
  volumes:
    - ./monitoring/prometheus/prometheus.yml:/etc/prometheus/prometheus.yml
    - prometheus-data:/prometheus
  command:
    - "--config.file=/etc/prometheus/prometheus.yml"
    - "--storage.tsdb.path=/prometheus"
    - "--web.enable-remote-write-receiver"
    - "--storage.tsdb.retention.time=7d"
  networks:
    - bench-net
  restart: unless-stopped

grafana:
  profiles: [scenario-a, scenario-b, scenario-c, scenario-d, scenario-e]
  image: grafana/grafana:latest
  ports:
    - "3000:3000"
  environment:
    GF_SECURITY_ADMIN_USER: admin
    GF_SECURITY_ADMIN_PASSWORD: benchmark
    GF_AUTH_ANONYMOUS_ENABLED: "true"
    GF_AUTH_ANONYMOUS_ORG_ROLE: Viewer
    GF_DASHBOARDS_DEFAULT_HOME_DASHBOARD_PATH:
        /var/lib/grafana/dashboards/benchmark.json
  volumes:
    - ./monitoring/grafana/provisioning:/etc/grafana/provisioning
    - ./monitoring/grafana/dashboards:/var/lib/grafana/dashboards
    - grafana-data:/var/lib/grafana
  networks:
    - bench-net
  depends_on:
    - prometheus
  restart: unless-stopped

# cAdvisor — expõe CPU/RAM dos containers em formato Prometheus
cadvisor:
  profiles: [scenario-a, scenario-b, scenario-c, scenario-d, scenario-e]
  image: gcr.io/cadvisor/cadvisor:latest
  ports:
```

```
      - "8081:8080"
    volumes:
      - /:/rootfs:ro
      - /var/run:/var/run:ro
      - /sys:/sys:ro
      - /var/lib/docker/:/var/lib/docker:ro
    networks:
      - bench-net
    restart: unless-stopped

networks:
  bench-net:
    driver: bridge

volumes:
  prometheus-data:
  grafana-data:
  postgres-data:
```

APÊNDICE B – IMAGENS E CONFIGURAÇÃO DOS SERVIDORES

O cenário E usa o mesmo `Dockerfile` do cenário B, trocando apenas o `Caddyfile` por `Caddyfile.default`.

docker/scenario-a/Dockerfile

```
FROM php:8.3.35-fpm-trixie@sha256:ed67b5fcd7600614dd651621a04214fbe16d373262cb31eb50bc22
    4c18cf0e88

# libicu-dev: necessário para ext-intl (requerida pelo CI4)
# libpq-dev: necessário para ext-pgsql/pdo_pgsql
# git/unzip: necessários para o Composer
RUN apt-get update \
    && apt-get install -y --no-install-recommends libicu-dev libpq-dev git unzip \
    && docker-php-ext-install opcache intl pgsql pdo_pgsql \
    && rm -rf /var/lib/apt/lists/*

# OPcache para benchmark justo (Scenario A recebe todos os otimizadores que tem direito)
COPY docker/scenario-a/opcache.ini /usr/local/etc/php/conf.d/opcache.ini

# Instalar composer
COPY --from=composer:2@sha256:af98f42dfff7c68ba8d53c2164fd9fde1087b7d449514baa38c418b1f6
    bc4bac /usr/bin/composer /usr/bin/composer

WORKDIR /var/www/html

# Copiar código da aplicação
COPY apps/ci4-base/ .

# Instalar dependências
RUN composer install --no-dev --optimize-autoloader --no-interaction --no-scripts

# Permissões de escrita para o CI4
RUN mkdir -p writable/{cache,logs,session,uploads,debugbar} \
    && chmod -R 777 writable/

# Variáveis de ambiente sobrescritas pelo docker-compose
ENV SCENARIO_NAME=scenario-a
ENV COMPUTE_SIZE=1000
ENV CI_ENVIRONMENT=production

EXPOSE 9000
CMD ["php-fpm"]
```

docker/scenario-a/php-fpm.conf

```
[www]
user  = www-data
group = www-data

listen = 0.0.0.0:9000

; Pool estático com 4 workers — mesma contagem dos Workers nos Cenários B/C/D
; Variável controlada nos 4 cenários para comparação justa
pm = static
pm.max_children = 4

; Desabilitar logs de acesso do FPM (reduz I/O nas medições)
access.log = /dev/null
```

docker/scenario-a/nginx.conf

```
server {
    listen 8080;
    root /var/www/html/public;
```

```
    index index.php;

    # Desabilita access log para eliminar I/O de disco das medições
    access_log off;
    error_log /var/log/nginx/error.log warn;

    location / {
        try_files $uri $uri/ /index.php?$query_string;
    }

    location ~ \.php$ {
        fastcgi_pass          app-a:9000;
        fastcgi_index         index.php;
        fastcgi_param         SCRIPT_FILENAME $document_root$fastcgi_script_name;
        fastcgi_read_timeout  60s;
        include               fastcgi_params;
    }

    location ~ /\. {
        deny all;
    }
}
```

docker/scenario-a/opcache.ini

```
opcache.enable=1
opcache.memory_consumption=128
opcache.interned_strings_buffer=8
opcache.max_accelerated_files=10000
opcache.validate_timestamps=0
opcache.revalidate_freq=0
opcache.save_comments=0
opcache.fast_shutdown=1
```

docker/scenario-b/Dockerfile

```
FROM dunglas/frankenphp:1.13.0-php8.3.35@sha256:3a1dfb9404aceb016493f6c18989cc2b66ce62c6
    de2a79ae6e3b3dfe7f94094a

# git necessário para o Composer em imagens Debian
RUN apt-get update && apt-get install -y --no-install-recommends git unzip && rm -rf
    /var/lib/apt/lists/*

# Extensões para CI4
RUN install-php-extensions opcache intl pgsql pdo_pgsql

# OPcache — mesmo nível de otimização do Cenário A para comparação justa
COPY docker/scenario-b/opcache.ini /usr/local/etc/php/conf.d/opcache.ini

# Instalar composer
COPY --from=composer:2@sha256:af98f42dfff7c68ba8d53c2164fd9fde1087b7d449514baa38c418b1f6
    bc4bac /usr/bin/composer /usr/bin/composer

WORKDIR /app

# Copiar código da aplicação CI4
COPY apps/ci4-base/ .

# Instalar dependências
RUN composer install --no-dev --optimize-autoloader --no-interaction --no-scripts

# Permissões de escrita para o CI4
RUN mkdir -p writable/{cache,logs,session,uploads,debugbar} \
    && chmod -R 777 writable/

# Caddyfile para modo Classic (sem diretiva worker — comportamento igual ao PHP-FPM)
COPY docker/scenario-b/Caddyfile /etc/frankenphp/Caddyfile
```

```
ENV SCENARIO_NAME=scenario-b
ENV COMPUTE_SIZE=1000
ENV CI_ENVIRONMENT=production
ENV SERVER_NAME=":8080"

EXPOSE 8080
```

docker/scenario-b/Caddyfile

```
{
    # FrankenPHP modo Classic — sem diretiva worker
    # Cada request inicia um novo contexto PHP (equivalente ao PHP-FPM)
    frankenphp {
        # 4 threads PHP: equivalente aos 4 processos do PHP-FPM (A) e aos 4 workers (C,
            D)
        num_threads 4
    }
    order php_server before file_server
    admin off
}

:8080 {
    root * /app/public

    php_server

    log {
        output discard
    }
}
```

docker/scenario-b/Caddyfile.default

```
{
    # FrankenPHP modo Classic — configuração PADRÃO de threads (sensibilidade; 2x os
        núcleos de CPU visíveis: 2 threads com o limite de 1,0 CPU)
    # Cada request inicia um novo contexto PHP (equivalente ao PHP-FPM)
    frankenphp
    order php_server before file_server
    admin off
}

:8080 {
    root * /app/public

    php_server

    log {
        output discard
    }
}
```

docker/scenario-c/Dockerfile

```
FROM dunglas/frankenphp:1.13.0-php8.3.35@sha256:3a1dfb9404aceb016493f6c18989cc2b66ce62c6
    de2a79ae6e3b3dfe7f94094a

# git necessário para o Composer em imagens Debian
RUN apt-get update && apt-get install -y --no-install-recommends git unzip && rm -rf
    /var/lib/apt/lists/*

# Extensões para CI4
RUN install-php-extensions opcache intl pgsql pdo_pgsql

# OPcache — mesmo nível dos outros cenários
COPY docker/scenario-c/opcache.ini /usr/local/etc/php/conf.d/opcache.ini

# Instalar composer
COPY --from=composer:2@sha256:af98f42dfff7c68ba8d53c2164fd9fde1087b7d449514baa38c418b1f6
```

```
    bc4bac /usr/bin/composer /usr/bin/composer

WORKDIR /app

# Copiar código da aplicação CI4
# Inclui public/frankenphp-worker.php (suporte nativo CI4 >= 4.7.0)
COPY apps/ci4-base/ .

# Instalar dependências (CI4 >= 4.7 para suporte nativo ao Worker Mode)
RUN composer install --no-dev --optimize-autoloader --no-interaction --no-scripts

# Gera public/frankenphp-worker.php via comando oficial do CI4
RUN php spark worker:install --no-interaction

# Permissões de escrita para o CI4
RUN mkdir -p writable/{cache,logs,session,uploads,debugbar} \
    && chmod -R 777 writable/

# Caddyfile oficial gerado pelo `php spark worker:install`
# Worker mode com 4 workers (mesmo número de pm.max_children do Cenário A)
COPY docker/scenario-c/Caddyfile /etc/frankenphp/Caddyfile

ENV SCENARIO_NAME=scenario-c
ENV COMPUTE_SIZE=1000
ENV CI_ENVIRONMENT=production
ENV SERVER_NAME=":8080"

EXPOSE 8080
```

docker/scenario-c/Caddyfile

```
{
    frankenphp {
        worker {
            file public/frankenphp-worker.php
            num {$FRANKENPHP_WORKERS:4}
        }
    }
    order php_server before file_server
    admin off
}

:8080 {
    root * public

    php_server {
        try_files {path} frankenphp-worker.php
    }

    file_server

    log {
        output discard
    }
}
```

docker/scenario-d/Dockerfile

```
FROM dunglas/frankenphp:1.13.0-php8.3.35@sha256:3a1dfb9404aceb016493f6c18989cc2b66ce62c6
    de2a79ae6e3b3dfe7f94094a

# git necessário para o Composer em imagens Debian
RUN apt-get update && apt-get install -y --no-install-recommends git unzip && rm -rf
    /var/lib/apt/lists/*

# Extensões para Laravel
RUN install-php-extensions opcache pcntl pgsql pdo_pgsql
```

```
# OPcache — mesmo nível dos outros cenários
COPY docker/scenario-d/opcache.ini /usr/local/etc/php/conf.d/opcache.ini


# Instalar composer
COPY --from=composer:2@sha256:af98f42dfff7c68ba8d53c2164fd9fde1087b7d449514baa38c418b1f6
    bc4bac /usr/bin/composer /usr/bin/composer


WORKDIR /app


# Copiar app Laravel
COPY apps/laravel-octane/ .


# Instalar dependências
RUN composer install --no-dev --optimize-autoloader --no-interaction --no-scripts


# Otimizações de produção do Laravel
RUN php artisan config:cache \
    && php artisan route:cache \
    && php artisan event:cache


# Permissões de armazenamento
RUN chmod -R 777 storage bootstrap/cache


ENV SCENARIO_NAME=scenario-d
ENV COMPUTE_SIZE=1000
ENV APP_ENV=production
ENV APP_DEBUG=false
ENV OCTANE_SERVER=frankenphp
ENV SERVER_NAME=":8080"


EXPOSE 8080


# Octane com FrankenPHP — 4 workers para comparação justa com os outros cenários
CMD ["php", "artisan", "octane:frankenphp", "--workers=4", "--port=8080",
    "--host=0.0.0.0"]
```

APÊNDICE C – SCRIPTS DE CARGA DO K6

k6/load.js

```
/**
 * Load Test — Teste de carga principal do TCC
 *
 * Distribuição de endpoints:
 *   10% health  — latência base
 *   30% compute — CPU-bound puro (sem I/O)
 *   30% memory  — persistência de estado em memória
 *   30% db      — persistência de conexão PostgreSQL (principal diferencial Worker
     Mode)
 *
 * Métricas via Prometheus remote_write → Grafana: http://localhost:3000
 *
 * Execução:
 *   docker compose --profile scenario-X run --rm k6 \
 *     run /scripts/load.js -e TARGET_URL=http://app-X:8080
 */

import { testHealth, testCompute, testMemory, testDb } from './lib/endpoints.js';

export const options = {
  stages: [
    { duration: '2m',  target: 10  },  // aquecimento
    { duration: '2m',  target: 100 },  // rampa de carga
    { duration: '10m', target: 100 },  // pico sustentado
    { duration: '1m',  target: 0   },  // desaceleração
  ],

  // Tag global: todas as métricas enviadas ao Prometheus carregam o label `scenario`.
  // Permite filtrar por cenário no Grafana mesmo com o TSDB acumulando múltiplos runs.
  tags: {
    scenario: __ENV.SCENARIO_NAME || 'unknown',
  },

  thresholds: {
    http_req_failed:             ['rate<0.05'],
    http_req_duration:           ['p(99)<5000'],
    'endpoint_health_duration':  ['p(95)<5000'],
    'endpoint_compute_duration': ['p(95)<5000'],
    'endpoint_memory_duration':  ['p(95)<5000'],
    'endpoint_db_duration':      ['p(95)<5000'],

    // Submétricas do PATAMAR (força o k6 a exportá-las no summary): RPS = contagem /
        duração do patamar
    'http_reqs{phase:plateau}':                    ['count>=0'],
    'http_req_failed{phase:plateau}':              ['rate<0.05'],
    'http_req_duration{phase:plateau}':            ['p(99)<60000'],
    'endpoint_health_duration{phase:plateau}':     ['p(99)<60000'],
    'endpoint_compute_duration{phase:plateau}':    ['p(99)<60000'],
    'endpoint_memory_duration{phase:plateau}':     ['p(99)<60000'],
    'endpoint_db_duration{phase:plateau}':         ['p(99)<60000'],
  },
};

export default function () {
  const rand = Math.random();

  if (rand < 0.10) {
    testHealth();
  } else if (rand < 0.40) {
    testCompute();
  } else if (rand < 0.70) {
    testMemory();
  } else {
    testDb();
  }
}
```

k6/long.js

```
/**
 * Long Test — mesmo perfil do load.js com patamar de 30 min (crescimento de memória)
 *
 * Distribuição de endpoints:
 *   10% health  — latência base
 *   30% compute — CPU-bound puro (sem I/O)
 *   30% memory  — persistência de estado em memória
 *   30% db      — persistência de conexão PostgreSQL (principal diferencial Worker
     Mode)
 *
 * Métricas via Prometheus remote_write → Grafana: http://localhost:3000
 *
 * Execução:
 *   docker compose --profile scenario-X run --rm k6 \
 *     run /scripts/load.js -e TARGET_URL=http://app-X:8080
 */

import { testHealth, testCompute, testMemory, testDb } from './lib/endpoints.js';

export const options = {
  stages: [
    { duration: '2m',  target: 10  },  // aquecimento
    { duration: '2m',  target: 100 },  // rampa de carga
    { duration: '30m', target: 100 },  // pico sustentado (teste longo de memória)
    { duration: '1m',  target: 0   },  // desaceleração
  ],

  // Tag global: todas as métricas enviadas ao Prometheus carregam o label `scenario`.
  // Permite filtrar por cenário no Grafana mesmo com o TSDB acumulando múltiplos runs.
  tags: {
    scenario: __ENV.SCENARIO_NAME || 'unknown',
  },

  thresholds: {
    http_req_failed:             ['rate<0.05'],
    http_req_duration:           ['p(99)<5000'],
    'endpoint_health_duration':  ['p(95)<5000'],
    'endpoint_compute_duration': ['p(95)<5000'],
    'endpoint_memory_duration':  ['p(95)<5000'],
    'endpoint_db_duration':      ['p(95)<5000'],

    // Submétricas do PATAMAR (força o k6 a exportá-las no summary): RPS = contagem /
        duração do patamar
    'http_reqs{phase:plateau}':                     ['count>=0'],
    'http_req_failed{phase:plateau}':               ['rate<0.05'],
    'http_req_duration{phase:plateau}':             ['p(99)<60000'],
    'endpoint_health_duration{phase:plateau}':      ['p(99)<60000'],
    'endpoint_compute_duration{phase:plateau}':     ['p(99)<60000'],
    'endpoint_memory_duration{phase:plateau}':      ['p(99)<60000'],
    'endpoint_db_duration{phase:plateau}':          ['p(99)<60000'],
  },
};

export default function () {
  const rand = Math.random();

  if (rand < 0.10) {
    testHealth();
  } else if (rand < 0.40) {
    testCompute();
  } else if (rand < 0.70) {
    testMemory();
  } else {
    testDb();
  }
}
```

k6/smoke.js

```
/**
```

```
 * Smoke Test — Sanidade do ambiente
 *
 * Execução:
 *   docker compose --profile scenario-X run --rm k6 \
 *     run /scripts/smoke.js -e TARGET_URL=http://app-X:8080
 */

import { sleep } from 'k6';
import { testHealth, testCompute, testMemory, testDb } from './lib/endpoints.js';

export const options = {
  vus:      1,
  duration: '1m',

  // Tag global: todas as métricas enviadas ao Prometheus carregam o label `scenario`.
  // Permite filtrar por cenário no Grafana mesmo com o TSDB acumulando múltiplos runs.
  tags: {
    scenario: __ENV.SCENARIO_NAME || 'unknown',
  },

  thresholds: {
    http_req_failed:             ['rate<0.01'],
    http_req_duration:           ['p(95)<500'],
    'endpoint_health_duration':  ['p(95)<100'],
    'endpoint_compute_duration': ['p(95)<1000'],
    'endpoint_memory_duration':  ['p(95)<200'],
    'endpoint_db_duration':      ['p(95)<300'],
  },
};

export default function () {
  testHealth();
  sleep(0.1);

  testCompute();
  sleep(0.1);

  testMemory();
  sleep(0.1);

  testDb();
  sleep(0.1);
}
```

k6/lib/endpoints.js

```
import http from 'k6/http';
import { check } from 'k6';
import { Trend } from 'k6/metrics';
import exec from 'k6/execution';

export const healthDuration  = new Trend('endpoint_health_duration',  true);
export const computeDuration = new Trend('endpoint_compute_duration', true);
export const memoryDuration  = new Trend('endpoint_memory_duration',  true);
export const dbDuration      = new Trend('endpoint_db_duration',      true);

const BASE_URL = __ENV.TARGET_URL || 'http://localhost:8080';

// Fase do teste, por tempo decorrido (s): warmup -> ramp -> plateau -> rampdown.
// Padrão = stages de load.js (2m + 2m + 10m + 1m). O patamar (plateau) é a janela de
    análise dos resultados.
const WARMUP_S  = Number(__ENV.WARMUP_S  || 120);
const RAMP_S    = Number(__ENV.RAMP_S    || 120);
const PLATEAU_S = Number(__ENV.PLATEAU_S || 600);
export function phase() {
  const t = exec.instance.currentTestRunDuration / 1000;
  if (t < WARMUP_S) return 'warmup';
  if (t < WARMUP_S + RAMP_S) return 'ramp';
  if (t < WARMUP_S + RAMP_S + PLATEAU_S) return 'plateau';
  return 'rampdown';
```

```
}

export function testHealth() {
  const ph = phase();
  const res = http.get(`${BASE_URL}/health`, { tags: { endpoint: 'health', phase: ph }
      });
  healthDuration.add(res.timings.duration, { phase: ph });
  check(res, {
    'health: status 200':         (r) => r.status === 200,
    'health: body has status ok': (r) => r.json('status') === 'ok',
  });
  return res;
}

export function testCompute() {
  const ph = phase();
  const res = http.get(`${BASE_URL}/compute`, { tags: { endpoint: 'compute', phase: ph }
      });
  computeDuration.add(res.timings.duration, { phase: ph });
  check(res, {
    'compute: status 200':          (r) => r.status === 200,
    'compute: body has data array': (r) => Array.isArray(r.json('data')),
  });
  return res;
}

export function testMemory() {
  const ph = phase();
  const res = http.get(`${BASE_URL}/memory`, { tags: { endpoint: 'memory', phase: ph }
      });
  memoryDuration.add(res.timings.duration, { phase: ph });
  check(res, {
    'memory: status 200':     (r) => r.status === 200,
    'memory: counter exists': (r) => r.json('counter') > 0,
    'memory: pid exists':     (r) => r.json('pid') > 0,
  });
  return res;
}

export function testDb() {
  const ph = phase();
  const res = http.get(`${BASE_URL}/db`, { tags: { endpoint: 'db', phase: ph } });
  dbDuration.add(res.timings.duration, { phase: ph });
  check(res, {
    'db: status 200':      (r) => r.status === 200,
    'db: item returned':   (r) => r.json('item') !== null,
    'db: item has id':     (r) => r.json('item.id') > 0,
  });
  return res;
}
```

APÊNDICE D – SCRIPTS DE AUTOMAÇÃO E ANÁLISE

scripts/run-completo.sh

```
#!/usr/bin/env bash
# run-completo.sh — Experimento completo, em uma única execução e sem intervenção.
# REGRA: sempre refaz TUDO do zero; qualquer falha aborta a execução inteira (descartar
    e recomeçar).
#
#  0) diagnóstico de persistência por worker (C e D, com 1 e 4 workers)
#  1) A, B, C, D x 3 (ordem rotativa)        -> results/runs/<c>-run<N>
#  2) E (B com threads padrão) x 3           -> results/runs/e-run<N>
#  3) testes longos (30 min no patamar): C e D -> results/runs/<c>-long
#  4) execuções de captura A-D com print do Grafana (tema claro, min 14:20, fora do
    patamar) -> results/captura
# Cada execução: contêineres anteriores derrubados, Prometheus zerado, cache do SO
    liberado (registro antes/depois), sem rebuild.
# Amostrador de CPU/RAM (docker stats, ~3 s) durante toda a coleta.
set -uo pipefail
cd "$(dirname "$0")/.."
export NO_BUILD=1
# Porta do hospedeiro para os testes manuais/diagnóstico (a 8080 pode estar ocupada por
    outro serviço da máquina)
export BENCH_PORT="${BENCH_PORT:-18080}"
(echo >/dev/tcp/127.0.0.1/"$BENCH_PORT") 2>/dev/null && { echo "ERRO: porta $BENCH_PORT
    já está em uso."; exit 1; }
for d in $( [[ -n "${CAPTURAS:-}" ]] && echo "" || echo "results/runs results/captura
    results/diagnostico" ); do
  [[ -z "$(ls -A "$d" 2>/dev/null)" ]] || { echo "ERRO: $d não está vazio. Arquive antes
      (mv); a coleta sempre recomeça do zero."; exit 1; }
done
echo 3 | sudo -n tee /proc/sys/vm/drop_caches >/dev/null 2>&1 || { echo "ERRO: sudo -n
    tee /proc/sys/vm/drop_caches não funciona (sudoers)."; exit 1; }
command -v google-chrome >/dev/null || { echo "ERRO: google-chrome ausente (necessário
    para os prints)."; exit 1; }
mkdir -p results/runs results/captura results/diagnostico docs/figuras
FLAG=$(mktemp); trap 'rm -f "$FLAG"' EXIT
falha() { echo "FALHA: $1 — coleta ABORTADA. Descarte results/ (arquive) e refaça
    TUDO."; rm -f "$FLAG"; exit 1; }
if [[ -z "${CAPTURAS:-}" ]]; then ./scripts/sample-stats.sh
    results/runs/stats-samples.csv "$FLAG" & fi


if [[ -z "${CAPTURAS:-}" ]]; then
echo "##### 0) DIAGNÓSTICO DE PERSISTÊNCIA #####"
for s in c d; do for w in 1 4; do ./scripts/diagnostico-persistencia.sh "$s" "$w" 200 ||
    falha "diagnóstico $s w=$w"; done; done


rodar() {  # <cenário> <teste> <rótulo> <duração-patamar-s>
  local s="$1" test="$2" dir="results/runs/$3"
  mkdir -p "$dir"; rm -f results/last-run.json
  PLATEAU_S="$4" ./scripts/run-scenario.sh "$s" "$test" 2>&1 | tee "$dir/console.log"
  grep -q "Teste concluído" "$dir/console.log" && [[ -f results/last-run.json ]] ||
      falha "$dir"
  docker stats --no-stream --format '{{.Name}},{{.CPUPerc}},{{.MemUsage}}' >
      "$dir/docker-stats-post.csv" 2>/dev/null || true
  mv results/last-run.json "$dir/"
  mv results/dashboard-scenario-${s}-${test}-*.html "$dir/" 2>/dev/null
  mv results/scenario-${s}-${test}-*.txt "$dir/" 2>/dev/null
  date -Is > "$dir/finished_at"
}
echo "##### 1) A,B,C,D x 3 #####"
for r in 1 2 3; do for s in a b c d; do rodar "$s" load "${s}-run${r}" 600; done; done
echo "##### 2) E x 3 #####"
for r in 1 2 3; do rodar e load "e-run${r}" 600; done
echo "##### 3) LONGOS C e D #####"
for s in c d; do rodar "$s" long "${s}-long" 1800; done


fi  # fim das etapas 0-3 (puladas ao retomar só as capturas com CAPTURAS="b c d")
echo "##### 4) CAPTURAS A-D #####"
URL='http://localhost:3000/d/tcc-uema-benchmark?kiosk&theme=light&from=now-16m&to=now'
for s in ${CAPTURAS:-a b c d}; do
  log=results/captura/console-$s.log
  ( until grep -q "running (14m2" "$log" 2>/dev/null; do sleep 2; done
```

```
    timeout 90 google-chrome --headless=new --no-sandbox --disable-gpu
        --user-data-dir=/tmp/claude-1000/chrome-prof \
      --window-size=1600,1140 --virtual-time-budget=20000
          --screenshot="docs/figuras/grafana-$s.png" "$URL" >/dev/null 2>&1 ) &
  CAP_PID=$!
  rm -f results/last-run.json
  PLATEAU_S=600 ./scripts/run-scenario.sh "$s" load > "$log" 2>&1
  wait "$CAP_PID"
  grep -q "Teste concluído" "$log" && [[ -s docs/figuras/grafana-$s.png ]] || falha
      "captura $s"
  mkdir -p results/captura/$s
  mv results/dashboard-scenario-$s-load-*.html results/scenario-$s-load-*.txt
      results/last-run.json results/captura/$s/ 2>/dev/null
done
docker compose --profile scenario-a --profile scenario-b --profile scenario-c --profile
    scenario-d --profile scenario-e down --remove-orphans
rm -f "$FLAG"; echo "COLETA COMPLETA"
```

scripts/run-scenario.sh

```
#!/usr/bin/env bash
# ================================================================================
# run-scenario.sh — Executa um cenário de benchmark de forma isolada e limpa
#
# Uso:
#   ./scripts/run-scenario.sh <cenário> <teste> [opções]
#
# Exemplos:
#   ./scripts/run-scenario.sh a smoke          # Cenário A, smoke test
#   ./scripts/run-scenario.sh b load           # Cenário B, load test
#   ./scripts/run-scenario.sh c load --no-cache-clear  # sem limpeza de cache
#
# Cenários disponíveis: a | b | c | d
# Testes disponíveis:   smoke | load | long | rate (taxa de chegada fixa; ver
    k6/rate.js)
# ================================================================================


set -euo pipefail


SCENARIO="${1:-}"
TEST="${2:-smoke}"
NO_CACHE_CLEAR=false


for arg in "${@:3}"; do
  [[ "$arg" == "--no-cache-clear" ]] && NO_CACHE_CLEAR=true
done


# Validações
if [[ -z "$SCENARIO" ]]; then
  echo "Erro: informe o cenário. Uso: $0 <a|b|c|d> <smoke|load>"
  exit 1
fi


if [[ ! "$SCENARIO" =~ ^[abcde]$ ]]; then
  echo "Erro: cenário inválido '$SCENARIO'. Use: a, b, c, d ou e"
  exit 1
fi


if [[ ! "$TEST" =~ ^(smoke|load|long|rate)$ ]]; then
  echo "Erro: teste inválido '$TEST'. Use: smoke, load, long ou rate"
  exit 1
fi


PROFILE="scenario-${SCENARIO}"


# Determina o host alvo do k6 (Cenário A usa nginx-a como proxy, demais usam app-X
    diretamente)
if [[ "$SCENARIO" == "a" ]]; then
  TARGET_HOST="nginx-a"
else
  TARGET_HOST="app-${SCENARIO}"
fi
```

```
TARGET_URL="http://${TARGET_HOST}:8080"

echo "========================================================================="
echo " TCC UEMA — Benchmark PHP Architecture"
echo " Cenário:  ${PROFILE^^} (${TARGET_HOST})"
echo " Teste:    ${TEST}"
echo " Data/hora: $(date '+%Y-%m-%d %H:%M:%S')"
echo "========================================================================="

# --- Passo 1: Derrubar qualquer cenário em execução e limpar métricas anteriores ---
echo ""
echo "[1/6] Derrubando cenários anteriores..."
docker compose \
  --profile scenario-a \
  --profile scenario-b \
  --profile scenario-c \
  --profile scenario-d \
  --profile scenario-e \
  down --remove-orphans 2>/dev/null || true

# Remove o volume do Prometheus para garantir que cada cenário comece com
# TSDB limpo. Sem isso, métricas de cenários anteriores ficam no banco e
# podem confundir a análise — especialmente em queries PromQL sem filtro de tempo.
# O grafana-data e postgres-data são preservados intencionalmente:
#   - grafana-data: dashboards e datasources provisionados não precisam resetar
#   - postgres-data: tabela read-only, dados determinísticos, init.sql já rodou
PROMETHEUS_VOLUME="$(basename "$(pwd)")_prometheus-data"
if docker volume inspect "$PROMETHEUS_VOLUME" &>/dev/null; then
  docker volume rm "$PROMETHEUS_VOLUME" > /dev/null
  echo "   Volume do Prometheus limpo (${PROMETHEUS_VOLUME})."
fi

# --- Passo 2: Limpar cache do sistema operacional ---
# Crítico para isolar os benchmarks: evita que resultados do cenário anterior
# influenciem o próximo por páginas de memória, dentries e inodes cacheados.
if [[ "$NO_CACHE_CLEAR" == false ]]; then
  echo ""
  echo "[2/6] Limpando cache do sistema operacional (page cache, dentries, inodes)..."
  if [[ "$(uname)" == "Linux" ]]; then
    sync
    echo "   Antes:  $(free -m | awk 'NR==2 {print "buff/cache=" $6 " MiB, livre=" $4 "
        MiB"}')"
    # sudo -n: nunca pede senha; exige a regra NOPASSWD para /usr/bin/tee
        /proc/sys/vm/drop_caches.
    # Se falhar, ABORTA (a limpeza faz parte do protocolo experimental e não pode ser
        pulada em silêncio).
    if echo 3 | sudo -n tee /proc/sys/vm/drop_caches > /dev/null 2>&1; then
      echo "   Depois: $(free -m | awk 'NR==2 {print "buff/cache=" $6 " MiB, livre=" $4
          " MiB"}')"
      echo "   Cache do SO liberado com sucesso."
    else
      echo "   ERRO: não foi possível limpar o cache do SO sem senha."
      echo "   Configure uma vez: echo \"$(whoami) ALL=(ALL) NOPASSWD: /usr/bin/tee
          /proc/sys/vm/drop_caches\" | sudo tee /etc/sudoers.d/drop-caches"
      echo "   (ou rode com --no-cache-clear para pular conscientemente)."
      exit 1
    fi
  elif [[ "$(uname)" == "Darwin" ]]; then
    sudo purge 2>/dev/null && echo "   Cache do macOS liberado (purge)." || { echo "
        ERRO: purge falhou."; exit 1; }
  else
    echo "   ERRO: sistema não reconhecido para limpeza de cache."; exit 1
  fi
else
  echo ""
  echo "[2/6] Limpeza de cache do SO ignorada (--no-cache-clear)."
fi

# --- Passo 3: Aguardar memória assentar ---
echo ""
echo "[3/6] Aguardando 3s para memória assentar..."
sleep 3

# --- Passo 4: Subir o cenário + stack de observabilidade ---
```

```
echo ""
echo "[4/6] Iniciando ${PROFILE} + Prometheus + Grafana + cAdvisor..."
if [[ "${NO_BUILD:-0}" == "1" ]]; then
  docker compose --profile "$PROFILE" up -d
else
  docker compose --profile "$PROFILE" up -d --build
fi

# --- Passo 5: Aguardar o app estar pronto ---
echo ""
echo "[5/6] Aguardando ${TARGET_HOST} responder em /health..."
MAX_ATTEMPTS=30
ATTEMPT=0
until docker compose --profile "$PROFILE" run --rm --no-deps k6 \
  run --quiet \
  --duration 1s --vus 1 \
  -e TARGET_URL="$TARGET_URL" \
  -e SCRIPT=smoke \
  /scripts/smoke.js &>/dev/null; do
  ATTEMPT=$((ATTEMPT + 1))
  if [[ $ATTEMPT -ge $MAX_ATTEMPTS ]]; then
    echo "   ERRO: ${TARGET_HOST} não respondeu após ${MAX_ATTEMPTS} tentativas."
    echo "   Verifique: docker compose --profile $PROFILE logs"
    exit 1
  fi
  echo "   Tentativa ${ATTEMPT}/${MAX_ATTEMPTS}... aguardando 2s"
  sleep 2
done
echo "   ${TARGET_HOST} está pronto!"

# --- Passo 6: Executar o teste k6 ---
echo ""
echo "[6/6] Executando ${TEST} test contra ${TARGET_URL}..."
echo "      Grafana disponível em http://localhost:3000 (admin / benchmark)"
echo ""

RESULTS_DIR="$(dirname "$0")/../results"
mkdir -p "$RESULTS_DIR"
TIMESTAMP="$(date '+%Y%m%d_%H%M%S')"
RESULTS_TXT="/results/scenario-${SCENARIO}-${TEST}-${TIMESTAMP}.txt"
DASHBOARD_HTML="/results/dashboard-${PROFILE}-${TEST}-${TIMESTAMP}.html"

docker compose --profile "$PROFILE" run --rm \
  -e K6_WEB_DASHBOARD=true \
  -e K6_WEB_DASHBOARD_EXPORT="$DASHBOARD_HTML" \
  k6 run \
  --out experimental-prometheus-rw \
  --summary-export /results/last-run.json \
  --summary-trend-stats="avg,min,med,max,p(90),p(95),p(99)" \
  -e TARGET_URL="$TARGET_URL" \
  -e SCENARIO_NAME="$PROFILE" \
  -e PLATEAU_S="${PLATEAU_S:-600}" \
  -e RATE="${RATE:-100}" \
  -e WARMUP_S="${WARMUP_S:-120}" \
  -e RAMP_S="${RAMP_S:-120}" \
  "/scripts/${TEST}.js" \
  | tee "${RESULTS_DIR}/scenario-${SCENARIO}-${TEST}-${TIMESTAMP}.txt"

echo ""
echo "================================================================="
echo " Teste concluído!"
echo " Resultados: ${RESULTS_DIR}/scenario-${SCENARIO}-${TEST}-${TIMESTAMP}.txt
 Dashboard:  ${RESULTS_DIR}/dashboard-${PROFILE}-${TEST}-${TIMESTAMP}.html"
echo " Grafana:    http://localhost:3000"
echo " Prometheus: http://localhost:9090"
echo "================================================================="
echo ""
echo " Para coletar snapshot de docker stats execute:"
echo "   ./scripts/collect-results.sh ${SCENARIO}"
echo ""
echo " Para derrubar o ambiente execute:"
echo "   docker compose --profile ${PROFILE} down"
echo "================================================================="
```

scripts/sample-stats.sh

```
#!/usr/bin/env bash
# sample-stats.sh — amostra docker stats (ts,container,cpu%,mem) a cada ~5 s até o
    arquivo-sinalizador sumir.
# Uso: sample-stats.sh <saida.csv> <arquivo-sinalizador>
OUT="${1:-results/runs/stats-samples.csv}"; FLAG="${2:-/tmp/sampling.flag}"
echo "ts,name,cpu_pct,mem_usage" > "$OUT"
while [[ -f "$FLAG" ]]; do
  ts=$(date +%s)
  docker stats --no-stream --format '{{.Name}},{{.CPUPerc}},{{.MemUsage}}' 2>/dev/null |
      grep -E 'app-[a-f]|nginx-a' | sed "s/^/$ts,/" >> "$OUT"
  sleep 3
done
```

scripts/diagnostico-persistencia.sh

```
#!/usr/bin/env bash
# Diagnóstico de persistência por worker: N requisições sequenciais e concorrentes a
    /memory e /db.
# Uso: ./scripts/diagnostico-persistencia.sh <c|d> <workers> [N=200]  ->
    results/diagnostico/<cenário>-w<workers>.json
set -uo pipefail
cd "$(dirname "$0")/.."
S="$1"; W="$2"; N="${3:-200}"; OUT=results/diagnostico; mkdir -p "$OUT"
docker compose --profile scenario-a --profile scenario-b --profile scenario-c --profile
    scenario-d --profile scenario-e down --remove-orphans >/dev/null 2>&1
if [[ "$S" == c ]]; then FRANKENPHP_WORKERS=$W docker compose --profile scenario-c up -d
    >/dev/null 2>&1
else OCTANE_WORKERS=$W docker compose --profile scenario-d up -d >/dev/null 2>&1; fi
for i in $(seq 1 60); do curl -sf localhost:${BENCH_PORT:-8080}/health >/dev/null &&
    break; sleep 1; done
curl -s localhost:${BENCH_PORT:-8080}/memory >/dev/null; curl -s
    localhost:${BENCH_PORT:-8080}/db >/dev/null   # aquecimento (descartado do reinício)
seqf=$(mktemp); conf=$(mktemp)
for i in $(seq 1 "$N"); do echo "$(curl -s localhost:${BENCH_PORT:-8080}/memory)"; echo
    "$(curl -s localhost:${BENCH_PORT:-8080}/db)"; done > "$seqf"
for i in $(seq 1 "$N"); do echo memory; echo db; done | xargs -P 16 -I{} curl -s -w '\n'
    localhost:${BENCH_PORT:-8080}/{} > "$conf"
python3 -I - "$seqf" "$conf" "$S" "$W" "$OUT/$S-w$W.json" <<'PY'
import sys,json,collections
sq,cf,s,w,out=sys.argv[1:6]
def parse(f):
    mem=collections.defaultdict(list); db=collections.defaultdict(set); n=0
    txt=open(f).read(); dec=json.JSONDecoder(); i=0
    while i<len(txt):
        while i<len(txt) and txt[i] in " \n": i+=1
        if i>=len(txt): break
        try: d,i=dec.raw_decode(txt,i)
        except Exception: i+=1; continue
        if not isinstance(d, dict): continue
        if 'counter' in d: mem[d['worker']].append(d['counter'])
        elif d.get('item'): db[d['worker']].add(d['item']['pg_pid'])
    return {k:{'n':len(v),'contador_min':min(v),'contador_max':max(v),'reinicios':sum(1
        for a,b in zip(v,v[1:]) if b<=a)} for k,v in mem.items()}, {k:sorted(v) for k,v
        in db.items()}
r={}
for nome,f in (("sequencial",sq),("concorrente",cf)):
    m,d=parse(f);
        r[nome]={"workers_distintos_memory":len(m),"memory":m,"db_pg_pids_por_worker":d}
if not any(r[k]['memory'] for k in r): sys.exit('diagnóstico sem dados (aplicação não
    respondeu)')
json.dump({"cenario":s,"workers":int(w),**r},open(out,'w'),indent=1,ensure_ascii=False)
for nome in ("sequencial","concorrente"):
    x=r[nome]; print(f"{s.upper()} workers={w} {nome}: workers
        vistos={x['workers_distintos_memory']}; por worker:",
        {k:(v['n'],v['contador_max'],v['reinicios']) for k,v in x['memory'].items()}, "|
        pg_pids por worker:", {k:len(v) for k,v in x['db_pg_pids_por_worker'].items()})
PY
docker compose --profile scenario-c --profile scenario-d down --remove-orphans
    >/dev/null 2>&1
```

scripts/analyze.py

```
#!/usr/bin/env python3
```

```
"""Agrega as rodadas em results/runs/<cenário>-run<N>/last-run.json (k6
    --summary-export)
e gera tabelas (Markdown/CSV) e figuras (PNG) em results/analysis/.

Uso: python3 scripts/analyze.py   (requer matplotlib para as figuras)
"""
import csv
import re
import json
import statistics as st
from pathlib import Path

ROOT = Path(__file__).resolve().parent.parent
RUNS = ROOT / "results" / "runs"
OUT = ROOT / "results" / "analysis"
OUT.mkdir(parents=True, exist_ok=True)

NOMES = {
    "a": "A — CI4 + PHP-FPM",
    "b": "B — CI4 + FrankenPHP clássico",
    "c": "C — CI4 + FrankenPHP Worker",
    "d": "D — Laravel Octane",
    "e": "E — CI4 + FrankenPHP clássico (threads padrão)",
}
PLATEAU_S = 600  # duração do patamar (s) nas execuções de 15 min
ENDPOINTS = ["health", "compute", "memory", "db"]
CORES = {"a": "#8c8c8c", "b": "#4c78a8", "c": "#e07b39", "d": "#59a14f", "e": "#b07aa1"}



def load():
    data = {s: [] for s in NOMES}
    for p in sorted(RUNS.glob("*-run*/last-run.json")):
        s = p.parent.name.split("-")[0]
        m = json.load(open(p))["metrics"]
        sf = "{phase:plateau}"  # patamar de 10 min (aquecimento, rampa e descida
            excluídos)
        dur = m[f"http_req_duration{sf}"]
        run = {
            "rps": m[f"http_reqs{sf}"]["count"] / PLATEAU_S,   # o campo `rate` da
                submétrica divide pela duração total
            "reqs": m[f"http_reqs{sf}"]["count"],
            "err": m[f"http_req_failed{sf}"].get("value",
                m[f"http_req_failed{sf}"].get("rate", 0)),
            "avg": dur["avg"],
            "p50": dur["med"],
            "p95": dur["p(95)"],
            "p99": dur.get("p(99)", float("nan")),
        }
        for e in ENDPOINTS:
            k = f"endpoint_{e}_duration{sf}"
            if k in m:
                run[f"{e}_avg"] = m[k]["avg"]
                run[f"{e}_p95"] = m[k]["p(95)"]
        data[s].append(run)
    return data



def recursos():
    """CPU/RAM por cenário a partir de results/runs/stats-samples.csv (docker stats a
        cada ~7 s).
    Janela do patamar sustentado de cada rodada: [fim-11,5 min, fim-1,5 min]. Cenário A
        soma app-a + nginx-a."""
    from datetime import datetime
    amostras = list(csv.DictReader(open(RUNS / "stats-samples.csv")))
    res = {s: [] for s in NOMES}
    for fin in sorted(RUNS.glob("*-run*/finished_at")):
        s = fin.parent.name.split("-")[0]
        if not (fin.parent / "last-run.json").exists():
            continue
        t1 = datetime.fromisoformat(fin.read_text().strip()).timestamp() - 90
        t0 = t1 - 600
        por_ts = {}
        for r in amostras:
```

```
            ts = int(r["ts"])
            if not (t0 <= ts <= t1) or not r["name"].startswith(f"tcc-uema-app-{s}") and
                not (s == "a" and "nginx-a" in r["name"]):
                continue
            cpu = float(r["cpu_pct"].rstrip("%"))
            mem = r["mem_usage"].split("/")[0].strip()
            val = float(re.sub("[^0-9.]", "", mem)) * (1024 if "GiB" in mem else 1)
            c, m = por_ts.get(ts, (0, 0))
            por_ts[ts] = (c + cpu, m + val)
        if por_ts:
            cs = [v[0] for v in por_ts.values()]
            ms_ = [v[1] for v in por_ts.values()]
            res[s].append({"cpu": st.mean(cs), "ram": st.mean(ms_), "ram_pico":
                max(ms_)})
    return res


def estatistica(data):
    """ANOVA de um fator e testes t de Welch par a par (n=3 por cenário: poder baixo,
        ver ressalva no texto)."""
    try:
        from scipy import stats
    except ImportError:
        print("scipy ausente: estatística não gerada")
        return
    pares = [("b", "a"), ("e", "b"), ("c", "a"), ("c", "b"), ("c", "e"), ("d", "c")]
    out = ["# Testes estatísticos (n=3 por cenário)", ""]
    for chave, rot in (("rps", "RPS"), ("p95", "Latência p95"), ("p99", "Latência
        p99")):
        grupos = [[r[chave] for r in data[s]] for s in data if len(data[s]) > 1]
        f, p = stats.f_oneway(*grupos)
        out += [f"## {rot}", f"ANOVA: F = {f:.1f}, p = {p:.2e}", "", "| Comparação |
            diferença (%) | t (Welch) | p | p ajustado (Bonferroni) |",
            "|---|---|---|---|---|"]
        for a_, b_ in pares:
            xa, xb = [r[chave] for r in data[a_]], [r[chave] for r in data[b_]]
            t, pv = stats.ttest_ind(xa, xb, equal_var=False)
            dif = (st.mean(xa) / st.mean(xb) - 1) * 100
            out.append(f"| {a_.upper()} vs {b_.upper()} | {dif:+.1f} | {t:.1f} |
                {pv:.2e} | {min(1, pv * len(pares)):.2e} |")
        out.append("")
    (OUT / "estatistica.md").write_text("\n".join(out))


def longos():
    """Teste de 30 min (c-long, d-long): RAM e CPU em janelas de 5 min do patamar, a
        partir de stats-samples.csv."""
    from datetime import datetime
    amostras = list(csv.DictReader(open(RUNS / "stats-samples.csv")))
    out = ["| Cenário | RPS | p95 (ms) | p99 (ms) | RAM 1ª janela (MiB) | RAM última
        janela (MiB) | RAM pico (MiB) |", "|---|---|---|---|---|---|---|"]
    series = {}
    for s in ("c", "d"):
        d = RUNS / f"{s}-long"
        if not (d / "last-run.json").exists():
            continue
        m = json.load(open(d / "last-run.json"))["metrics"]
        fim = datetime.fromisoformat((d /
            "finished_at").read_text().strip()).timestamp()
        ini = fim - 35 * 60 + 4 * 60   # início do patamar (2 min + 2 min de rampa)
        fimp = fim - 60                # fim do patamar
        pts = []
        for r in amostras:
            ts = int(r["ts"])
            if r["name"] != f"tcc-uema-app-{s}-1" or not (ini <= ts <= fimp):
                continue
            mem = r["mem_usage"].split("/")[0].strip()
            pts.append(((ts - ini) / 60, float(re.sub("[^0-9.]", "", mem)) * (1024 if
                "GiB" in mem else 1),
                        float(r["cpu_pct"].rstrip("%"))))
        series[s] = pts
        if not pts:
            continue
        prim = [p[1] for p in pts if p[0] <= 5]
        ult = [p[1] for p in pts if p[0] >= 25]
```

```
        sf = "{phase:plateau}"  # patamar de 30 min
        out.append(f"| {NOMES[s]} | {m[f'http_reqs{sf}']['count'] / 1800:.1f} |
            {m[f'http_req_duration{sf}']['p(95)']:.1f} | "
                   f"{m[f'http_req_duration{sf}']['p(99)']:.1f} | {st.mean(prim):.1f} |
                       {st.mean(ult):.1f} | {max(p[1] for p in pts):.1f} |")
    (OUT / "tabela_longos.md").write_text("\n".join(out) + "\n")
    with open(OUT / "longos_ram.csv", "w", newline="") as f:
        w = csv.writer(f)
        w.writerow(["cenario", "minuto_patamar", "ram_mib", "cpu_pct"])
        for s, pts in series.items():
            for t, ram, cpu in pts:
                w.writerow([s, f"{t:.2f}", ram, cpu])
    globals()["_SERIES"] = series



def ms(vals):
    vals = [v for v in vals if v == v]
    if not vals:
        return (float("nan"), float("nan"))
    return (st.mean(vals), st.stdev(vals) if len(vals) > 1 else 0.0)



def main():
    data = load()
    agg = {}
    for s, runs in data.items():
        if not runs:
            continue
        agg[s] = {k: ms([r[k] for r in runs if k in r]) for k in runs[0]}
        agg[s]["n"] = len(runs)

    # Tabela 1/2 (CSV + Markdown)
    linhas = ["| Cenário | n | RPS (média ± dp) | Latência p50 (ms) | p95 (ms) | p99
        (ms) | Erros (%) |",
              "|---|---|---|---|---|---|---|"]
    with open(OUT / "global.csv", "w", newline="") as f:
        w = csv.writer(f)
        w.writerow(["cenario", "n", "rps_media", "rps_dp", "p50", "p95", "p99",
            "erro_pct"])
        for s, a in agg.items():
            linhas.append(
                f"| {NOMES[s]} | {a['n']} | {a['rps'][0]:.1f} ± {a['rps'][1]:.1f} |
                    {a['p50'][0]:.1f} | "
                f"{a['p95'][0]:.1f} | {a['p99'][0]:.1f} | {a['err'][0]*100:.2f} |")
            w.writerow([s, a["n"], a["rps"][0], a["rps"][1], a["p50"][0], a["p95"][0],
                a["p99"][0], a["err"][0] * 100])
    (OUT / "tabela_global.md").write_text("\n".join(linhas) + "\n")

    linhas = ["| Cenário | " + " | ".join(f"{e} avg / p95 (ms)" for e in ENDPOINTS) + "
        |",
              "|---|" + "---|" * len(ENDPOINTS)]
    for s, a in agg.items():
        cel = [f"{a[f'{e}_avg'][0]:.1f} / {a[f'{e}_p95'][0]:.1f}" for e in ENDPOINTS]
        linhas.append(f"| {NOMES[s]} | " + " | ".join(cel) + " |")
    (OUT / "tabela_endpoints.md").write_text("\n".join(linhas) + "\n")

    # Tabela 3: recursos
    rec = recursos()
    linhas = ["| Cenário | CPU média (%) | RAM média (MiB) | RAM pico (MiB) |",
        "|---|---|---|---|"]
    ragg = {}
    for s, rr in rec.items():
        if rr:
            ragg[s] = {k: ms([r[k] for r in rr]) for k in ("cpu", "ram", "ram_pico")}
            a = ragg[s]
            linhas.append(f"| {NOMES[s]} | {a['cpu'][0]:.1f} ± {a['cpu'][1]:.1f} |
                {a['ram'][0]:.1f} ± {a['ram'][1]:.1f} | {a['ram_pico'][0]:.1f} |")
    (OUT / "tabela_recursos.md").write_text("\n".join(linhas) + "\n")

    # Ganhos relativos de RPS
    ganhos = []
    for a_, b_ in [("b", "a"), ("e", "b"), ("c", "a"), ("c", "b"), ("c", "e"), ("d",
        "c"), ("d", "a")]:
```

```
        if a_ in agg and b_ in agg:
            g = (agg[a_]["rps"][0] / agg[b_]["rps"][0] - 1) * 100
            ganhos.append(f"{a_.upper()} vs {b_.upper()}: {g:+.1f}% RPS")
    (OUT / "ganhos.txt").write_text("\n".join(ganhos) + "\n")

    estatistica(data)
    longos()

    # Figuras
    try:
        import matplotlib
        matplotlib.use("Agg")
        import matplotlib.pyplot as plt
    except ImportError:
        print("matplotlib ausente: figuras não geradas")
        return
    plt.rcParams.update({"font.size": 11, "font.family": "DejaVu Sans"})

    ss = list(agg)
    fig, ax = plt.subplots(figsize=(7, 4))
    ax.bar([NOMES[s].split(" — ")[0] for s in ss], [agg[s]["rps"][0] for s in ss],
           yerr=[agg[s]["rps"][1] for s in ss], color=[CORES[s] for s in ss], capsize=4)
    for i, s in enumerate(ss):
        ax.text(i, agg[s]["rps"][0], f"{agg[s]['rps'][0]:.0f}", ha="center",
            va="bottom")
    ax.set_ylabel("Requisições por segundo (média ± dp)")
    ax.set_xlabel("Cenário")
    fig.tight_layout()
    fig.savefig(OUT / "fig_rps.png", dpi=200)
    plt.close(fig)

    fig, ax = plt.subplots(figsize=(8, 4.2))
    w = 0.8 / len(ss)
    for i, s in enumerate(ss):
        ax.bar([j + i * w for j in range(len(ENDPOINTS))], [agg[s][f"{e}_p95"][0] for e
            in ENDPOINTS],
               w, label=NOMES[s].split(" — ")[0], color=CORES[s])
    ax.set_xticks([j + 0.4 - w / 2 for j in range(len(ENDPOINTS))])
    ax.set_xticklabels(ENDPOINTS)
    ax.set_ylabel("Latência p95 (ms)")
    ax.set_xlabel("Endpoint")
    ax.legend(title="Cenário")
    fig.tight_layout()
    fig.savefig(OUT / "fig_latencia_p95.png", dpi=200)
    plt.close(fig)
    if ragg:
        fig, axs = plt.subplots(1, 2, figsize=(9, 3.8))
        ks = list(ragg)
        for ax, k, t in ((axs[0], "cpu", "CPU média (%)"), (axs[1], "ram", "RAM média
            (MiB)")):
            ax.bar([NOMES[s].split(" — ")[0] for s in ks], [ragg[s][k][0] for s in ks],
                   yerr=[ragg[s][k][1] for s in ks], color=[CORES[s] for s in ks],
                       capsize=4)
            ax.set_ylabel(t)
            ax.set_xlabel("Cenário")
        fig.tight_layout()
        fig.savefig(OUT / "fig_recursos.png", dpi=200)
        plt.close(fig)
    if globals().get("_SERIES"):
        fig, ax = plt.subplots(figsize=(7.5, 4))
        for s, pts in _SERIES.items():
            ax.plot([p[0] for p in pts], [p[1] for p in pts], color=CORES[s],
                label=NOMES[s].split(" — ")[0], lw=1.2)
        ax.set_xlabel("Tempo no patamar (min)")
        ax.set_ylabel("RAM do contêiner (MiB)")
        ax.set_ylim(bottom=0)
        ax.legend(title="Cenário")
        fig.tight_layout()
        fig.savefig(OUT / "fig_ram_longo.png", dpi=200)
        plt.close(fig)
    print("OK:", OUT)


if __name__ == "__main__":
```

```
main()
```

APÊNDICE E – CONTROLADORES DOS ENDPOINTS

apps/ci4-base/app/Controllers/BenchmarkController.php

```
<?php

namespace App\Controllers;

use CodeIgniter\Controller;

class BenchmarkController extends Controller
{
    public function health(): void
    {
        $this->response->setHeader('Content-Type', 'application/json');
        echo json_encode([
            'status'   => 'ok',
            'scenario' => getenv('SCENARIO_NAME') ?: 'unknown',
        ]);
    }

    public function compute(): void
    {
        $size = (int) (getenv('COMPUTE_SIZE') ?: 1000);
        $data = [];

        for ($i = 0; $i < $size; $i++) {
            $data[] = [
                'id'    => $i + 1,
                'uuid'  => md5(uniqid((string) $i, true)),
                'value' => random_int(1, 999999),
                'label' => str_repeat('x', random_int(5, 20)),
            ];
        }

        $this->response->setHeader('Content-Type', 'application/json');
        echo json_encode([
            'count'    => $size,
            'scenario' => getenv('SCENARIO_NAME') ?: 'unknown',
            'data'     => $data,
        ]);
    }

    public function memory(): void
    {
        // Propriedade estática sobrevive ao loop do Worker Mode (mesma instância PHP)
        // Em PHP-FPM/Classic: reinicia a cada request (processo novo ou sem estado
            persistente)
        MemoryState::$counter++;
        MemoryState::$worker ??= bin2hex(random_bytes(4));

        $this->response->setHeader('Content-Type', 'application/json');
        echo json_encode([
            'counter'  => MemoryState::$counter,
            'worker'   => MemoryState::$worker,
            'pid'      => getmypid(),
            'scenario' => getenv('SCENARIO_NAME') ?: 'unknown',
        ]);
    }

    public function db(): void
    {
        $id = random_int(1, 10000);

        // pg_backend_pid() retorna o PID do processo PostgreSQL que serve esta conexão.
        // Worker Mode (C, D): pg_pid constante = mesma conexão reutilizada entre
            requests.
        // PHP-FPM/Classic (A, B): pg_pid varia = nova conexão estabelecida por request.
        // Incluído na mesma query — sem round-trip extra.
        $row = db_connect()
```

```
            ->query(
                'SELECT id, name, value, pg_backend_pid() AS pg_pid FROM benchmark_items
                    WHERE id = ?',
                [$id]
            )
            ->getRow();

        $this->response->setHeader('Content-Type', 'application/json');
        echo json_encode([
            'item'     => $row,
            'php_pid'  => getmypid(),
            'worker'   => (MemoryState::$worker ??= bin2hex(random_bytes(4))),
            'scenario' => getenv('SCENARIO_NAME') ?: 'unknown',
        ]);
    }
}

class MemoryState
{
    public static int $counter = 0;
    public static ?string $worker = null; // identifica o worker (thread) que atende a
        requisição
}
```

apps/laravel-octane/app/Http/Controllers/BenchmarkController.php

```
<?php

namespace App\Http\Controllers;

use Illuminate\Http\JsonResponse;
use Illuminate\Routing\Controller;
use Illuminate\Support\Facades\DB;

class BenchmarkController extends Controller
{
    private static int $counter = 0;
    private static ?string $worker = null; // identifica o worker que atende a
        requisição

    public function health(): JsonResponse
    {
        return response()->json([
            'status'   => 'ok',
            'scenario' => env('SCENARIO_NAME', 'scenario-d'),
        ]);
    }

    public function compute(): JsonResponse
    {
        $size = (int) env('COMPUTE_SIZE', 1000);
        $data = [];

        for ($i = 0; $i < $size; $i++) {
            $data[] = [
                'id'    => $i + 1,
                'uuid'  => md5(uniqid((string) $i, true)),
                'value' => random_int(1, 999999),
                'label' => str_repeat('x', random_int(5, 20)),
            ];
        }

        return response()->json([
            'count'    => $size,
            'scenario' => env('SCENARIO_NAME', 'scenario-d'),
            'data'     => $data,
        ]);
    }

    public function memory(): JsonResponse
    {
```

```
        self::$counter++;
        self::$worker ??= bin2hex(random_bytes(4));

        return response()->json([
            'counter'  => self::$counter,
            'worker'   => self::$worker,
            'pid'      => getmypid(),
            'scenario' => env('SCENARIO_NAME', 'scenario-d'),
        ]);
    }

    public function db(): JsonResponse
    {
        $id = random_int(1, 10000);

        // pg_backend_pid() incluído na mesma query — sem round-trip extra.
        // Mesmo papel do endpoint /memory: prova persistência, mas agora
        // no nível da conexão PostgreSQL em vez do estado PHP em memória.
        $row = DB::selectOne(
            'SELECT id, name, value, pg_backend_pid() AS pg_pid FROM benchmark_items
                WHERE id = ?',
            [$id]
        );

        return response()->json([
            'item'     => $row,
            'php_pid'  => getmypid(),
            'worker'   => (self::$worker ??= bin2hex(random_bytes(4))),
            'scenario' => env('SCENARIO_NAME', 'scenario-d'),
        ]);
    }
}
```

APÊNDICE F – RESULTADOS POR EXECUÇÃO

Tabela 6 – Resultados de cada uma das 15 execuções de 15 minutos

Execução	RPS	Latência p50 (ms)	Latência p95 (ms)	Latência p99 (ms)	Erros (%)
A – execução 1	144,1	686,5	782,9	797,3	0,00
A – execução 2	143,2	688,2	787,5	802,7	0,00
A – execução 3	143,9	687,6	786,4	801,4	0,00
B – execução 1	154,7	605,3	710,4	792,6	0,00
B – execução 2	157,5	600,3	704,2	766,5	0,00
B – execução 3	156,3	602,5	715,5	789,5	0,00
C – execução 1	314,3	301,4	394,8	406,8	0,00
C – execução 2	314,9	301,2	394,1	405,4	0,00
C – execução 3	325,2	297,9	393,4	404,7	0,00
D – execução 1	350,4	287,6	375,2	397,0	0,00
D – execução 2	352,2	287,9	375,5	396,8	0,00
D – execução 3	351,3	287,2	373,2	396,5	0,00
E – execução 1	175,8	524,3	658,5	710,1	0,00
E – execução 2	173,6	525,4	675,8	742,1	0,00
E – execução 3	177,0	524,2	647,4	709,8	0,00

Fonte: elaborado pelo próprio autor (2026). Nota: estatísticas sobre toda a execução, incluindo as rampas.

Tabela 7 – Resultados dos testes longos de 30 minutos de patamar

Teste	RPS	Latência p50 (ms)	Latência p95 (ms)	Latência p99 (ms)	Erros (%)
C – teste longo (30 min)	310,1	306,3	396,1	406,7	0,00
D – teste longo (30 min)	346,8	293,9	376,5	397,0	0,00

Fonte: elaborado pelo próprio autor (2026).

APÊNDICE G – AMBIENTE DE EXECUÇÃO

Saída dos comandos de inspeção do hospedeiro, registrada pelo autor antes da coleta.

```
Linux andre-vaio 7.0.0-38-generic #38-Ubuntu SMP PREEMPT_DYNAMIC Fri Sep  4 09:10:14 UTC
    2026 x86_64 GNU/Linux
CPU(s):                                16
Nome do modelo:                        AMD Ryzen 7 5700U with Radeon Graphics
               total       usada       livre    compart.  buff/cache  disponível
Mem.:           14Gi       8,0Gi       5,1Gi       347Mi       2,1Gi       6,5Gi
Swap:          4,0Gi       2,7Gi       1,3Gi
Docker version 28.1.1, build 4eba377
Docker Compose version v2.31.0-desktop.2
FrankenPHP v1.13.0 PHP 8.3.35 Caddy v2.11.7 h1:yj0Y4fYZGPkSv
PHP 8.3.35 (cli) (built: Oct  6 2026 01:29:10) (ZTS)
PHP 8.3.35 (cli) (built: Sep 25 2026 17:44:10) (NTS)
```